

Cascading Style Sheet (CSS)

Internet Programming I: Chapter 3 – Part I

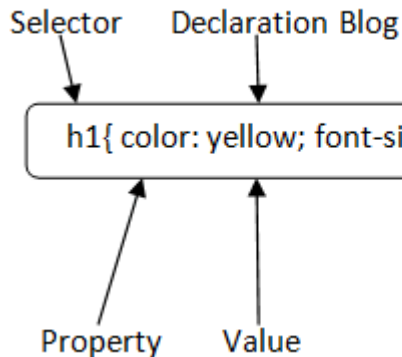


ADDIS ABABA SCIENCE AND TECHNOLOGY UNIVERSITY
Department of Software Engineering

Introduction to CSS



- CSS is a styling language
- It is used to describe the look and formatting of a document written in markup language
- CSS Syntax
 - **Selector:** indicates the HTML element you want to style
 - any tag like <h1>, <title> etc
 - **Declaration Block:** can contain one or more declarations separated by a semicolon
 - color: yellow;
 - font-size: 11 px;
 - Each declaration contains a property name and value, separated by a colon
 - **Property:** A Property is a type of attribute of HTML element
 - E.g. color, border etc.
 - **Value:** Values are assigned to CSS properties
 - E.g. value "yellow" is assigned to color property



CSS Selectors

- **Element Selector:** selects the HTML element by name
 - E.g.

```
p{
    text-align: center;
    color: blue;
}
```
- **Id Selector:** selects the id attribute of an HTML element to select a specific element
 - It is written with the hash character (#), followed by the id of the element
- **Universal selector (*):** used as a wildcard character. It selects all the elements on the pages.



```
<html>
<head>
<style>
* {
    color: green;
    font-size: 20px;
}
#para1 {
    text-align: center;
    color: blue;
}
</style>
</head>
<body>
<p id="para1">Hello Javatpoint.com</p>

<p> This paragraph will not be affected.
</p>
</body>
</html>
```

CSS Selectors cont'd

- **Class selector:** selects HTML elements with a specific class attribute
 - It is used with a period character . (full stop symbol) followed by the class name
- Use element name with class selector to affect specific element
- **Group selector:** used to select all the elements with the same style definitions and minimize the code
 - Commas are used to separate each selector in grouping



```
<html>
<head>
<style>
.center {
    text-align: center;
    color: blue; }
p.center {
    text-align: center;
    color: blue; }
h1,h2,p {
    text-align: center;
    color: blue; }
</style>
</head>
<body>
<h1 class="center">This heading is blue and
center-aligned.</h1>
<p class="center">This paragraph is blue an
d center-aligned.</p>
</body>
</html>
```

Adding CSS



- There are three ways to insert CSS in HTML documents
 1. **Inline CSS:** used to apply CSS on a single line or element
 - E.g. `<p style="color:blue">Hello CSS</p>`
 2. **Internal CSS:** used to apply CSS on a single document or page
 - It can affect all the elements of the page. It is written inside the style tag within head section of html

```
<style>
p{color:blue}
</style>
```

3. **External CSS:** used to apply CSS on multiple pages or all pages
 - Here, we write all the CSS code in a CSS file with .css extension example style.css

style.css

```
p{color:blue}
```

to link this style.css file to the html pages

```
<link rel="stylesheet" type="text/css" href="style.css">
```

Disadvantages of Inline CSS



- You cannot use quotations within inline CSS
 - If you use quotations the browser will interpret this as an end of your style value
- These styles cannot be reused anywhere else
- These styles are tough to be edited because they are not stored at a single place
- It is not possible to style pseudo-codes and pseudo-classes with inline CSS
- Inline CSS does not provide browser cache advantages

Comments



- CSS comments are written within `/*.....*/`

```
<style>
p {
  color: blue;
  /* This is a single-line comment */
  text-align: center;
}
/* This is
a multi-line
comment */
</style>
```

CSS Background



- Background property is used to define the background effects on element
- **Background-color:** used to set the background color of an element

Syntax

```
element {  
  background-color: color name | transparent | initial | inherit;  
}
```

- Example

```
background-color: #988989;
```

```
background-color: #988989;
```

- **Background-image:** used to set an image as a background of an element

```
<!DOCTYPE html>  
<html>  
<head>  
<style>  
  body {  
    background-image: url("paper1.gif");  
    margin-left: 100px;  
  }  
</style>  
</head>  
<body>  
<h1>Hello World </h1>  
</body>  
</html>
```



CSS Background cont'd



- **Background-repeat:** by default, the background-image property repeats the background image horizontally and vertically
 - background-repeat: repeat-x; => background image repeated horizontally only
 - background-repeat: repeat-y; => background image repeated vertically only
 - background-repeat: no-repeat; => stop background repeat
- **Background-attachment:**
 - used to specify that the background image is fixed or scroll with the rest of the page in the browser window
 - **scroll:** It is the default value that prevents the element from scrolling with the contents, but scrolls with the page
 - **fixed:** the background image doesn't move with the element
 - **local:** if the element has a scrolling mechanism, the background image scrolls with the content of the element
 - **initial:** It sets the property to its default value
 - **inherit:** It inherits the property from its parent element

Syntax

```
background-attachment: scroll | fixed | local | initial | inherit;
```

CSS Background cont'd



- **Background-size:** used to set the size of a background image of an element
 - Can be defined using length, percentage, or keyword values

Syntax

```
background-size: auto | length | cover | contain | initial | inherit;
```

- **auto:** the default value, which displays the background image in its original size
- **length:** set width and height of background image
- **percentage:** defines width and height of the background image to the percentage (%) of the background positioning area
- **cover:** used to resize the background image to cover the entire container
- **contain:** Without stretching or cropping, it resizes the background image to ensure the image is completely visible
- **initial:** It sets the property to its default value
- **inherit:** It inherits the property from its parent element

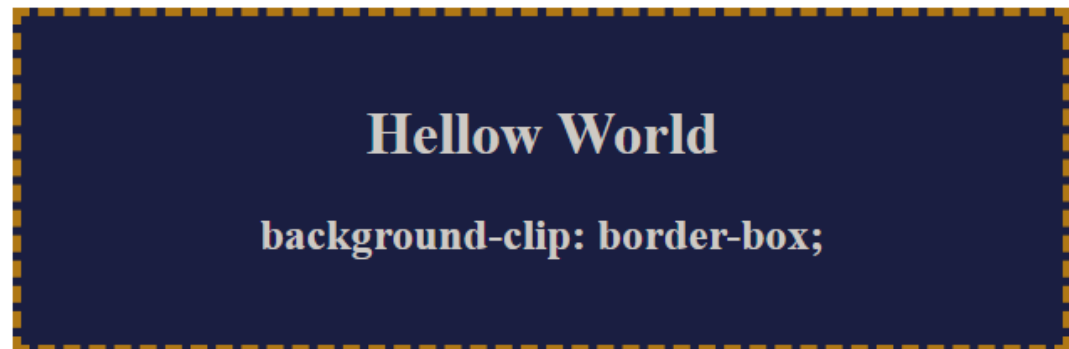
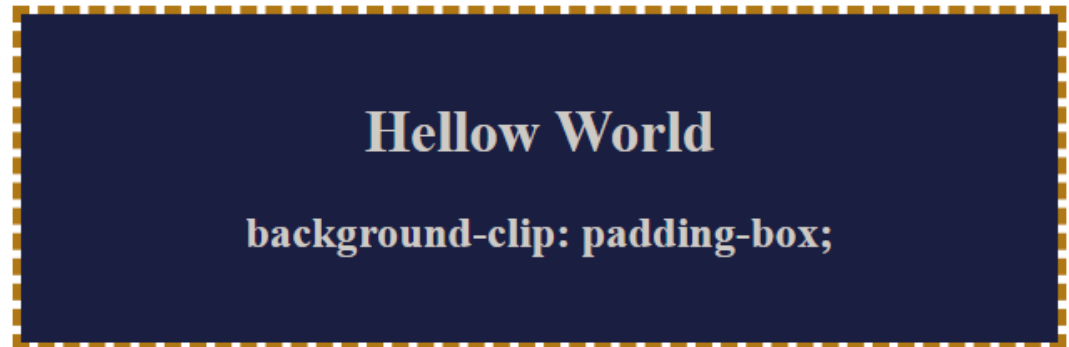
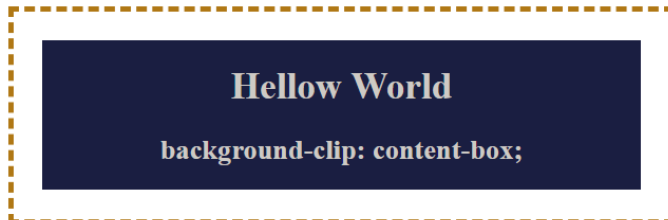
Example

```
#div1{  
  background-image: url('lion.png');  
  background-size: auto; }  
#div2{  
  background-image: url('lion.png');  
  background-size: 150px 150px; }  
#div3{  
  background-image: url('lion.png');  
  background-size: 30%; }
```

CSS Background cont'd



- **Background-clip:** limits the area in which the background color or image appears by applying a clipping box
 - Anything outside the box will be discarded and invisible
- Syntax: `background-clip: border-box | padding-box | content-box | inherit;`
- Example



CSS Background cont'd

- **Background-blend-mode:** defines how the background image of an element blends with the background color of the element
 - We can blend the background images together or can blend them with background-color
- Syntax: background-blend-mode: normal | multiply | screen | color-dodge | difference | darken | lighten | saturation | luminosity | overlay | hard-light | soft-light | exclusion | hue | color-burn | color;

Original Image: background: url("image1.png"), url("image2.png");



background-blend-mode: multiply;



background-blend-mode: screen;



background-blend-mode: color-dodge;



CSS Background cont'd

- Background-blend-mode

background-blend-mode: difference;



background-blend-mode: overlay;



background-blend-mode: lighten;



background-blend-mode: luminosity;



background-blend-mode: darken;



background-blend-mode: saturation;



CSS Background cont'd

- **Background-blend-mode**

`background-blend-mode: hard-light;`



`background-blend-mode: soft-light;`



`background-blend-mode: exclusion;`



`background-blend-mode: hue;`



`background-blend-mode: color-burn;`



`background-blend-mode: color;`



CSS Background cont'd

- **background-origin:** It specifies the background position area, i.e., the origin of a background image
 - This CSS property will not work when the value of the background-attachment is set to be fixed
- The background-origin property is similar to the background-clip property, but it resizes the background instead of clipping it
- By default, the origin of an element is the top-left corner of the screen
- Syntax: background-origin: padding-box | border-box | content-box | initial | inherit;

background-origin: border-box;



background-origin: padding-box;



background-origin: content-box;



CSS Border



- **Border style:** used to specify the border type which you want to display on the web page
- Syntax: border-style: solid | groove ridge inset outset double dotted dashed none;

No border.

A dotted border.

A dashed border.

A solid border.

A double border.

A groove border.

A ridge border.

An inset border.

An outset border.

A hidden border.

CSS **Border** cont'd



- **border-width:** used to set the border's width in pixels
 - You can also use one of the three pre-defined values, **thin**, **medium** or **thick** to set the width of the border
- Example:
 - `border-width: medium;`
 - `border-width: 5px;`
- **border-color:** three methods to set the color of the border
 - Name: e.g. `border-color: red;`
 - RGB: e.g. `border-color: rgb(34, 44, 233);`
 - Hex: e.g. `border-color: #98bf21;`

CSS **Border** cont'd



- **border-radius:** sets the rounded borders and provides the rounded corners around an element, tags, or div
- border-radius is shorthand for border top-left-radius, border-top-right-radius, border-bottom-right-radius and border-bottom-left-radius
- We can specify the border for all four corners of the box in a single declaration using the border-radius
- The values of this property can be defined in percentage or length units
- Syntax
 - border-radius: 1-4 length | % / 1-4 length | % | inherit | initial;

CSS Border cont'd



**Welcome to the
javaTpoint.com**

`border-radius: 90px;`

**Welcome to the
javaTpoint.com**

`border-radius: 25% 10%;`

**Welcome to the
javaTpoint.com**

`border-top-left-radius: 250px;`

**Welcome to the
javaTpoint.com**

`border-radius: 35px 10em
10%;`

**Welcome to the
javaTpoint.com**

`border-radius: 50px 50%
50cm 50em;`

**Welcome to the
javaTpoint.com**

`border-bottom-right-radius:
50%;`

CSS **Border** cont'd



- **border-radius:** We can specify separate horizontal and vertical values by using the slash (/) symbol
 - The values before the slash (/) is used for the horizontal radius and
 - The values after the slash (/) are for the vertical radius



CSS Border cont'd



- **border-collapse:** used to set the border of the table cells
- specifies whether the table cells share the separate or common border
- This property has two main values that are separate and collapse
 - **Separate:** the distance between the cells can be defined using the border-spacing property
 - **Collapse:** then the inset value of border-style property behaves like groove, and the outset value behaves like ridge
- Syntax
 - border-collapse: separate | collapse | initial | inherit;

The border-collapse Property

border-collapse: separate;

First_Name	Last_Name	Subject	Marks
James	Gosling	Maths	92
Alan	Rickman	Maths	89
			82

#t1{ border-collapse: separate; }

The border-collapse Property

border-collapse: collapse;

First_Name	Last_Name	Subject	Marks
James	Gosling	Maths	92
Alan	Rickman	Maths	89
Sam			

#t2{ border-collapse: collapse; }

CSS Border cont'd



- **border-spacing:** used to set the distance between the borders of the adjacent cells in the table
 - It applies only when the border-collapse property is set to separate
- It can be defined as one or two values for determining the vertical and horizontal spacing.
 - one value sets both horizontal and vertical spacing
 - two-value, the first one is used to set the horizontal spacing, and the second value sets the vertical spacing
- Syntax
 - border-spacing: length | initial | inherit;

The border-spacing Property

border-spacing: 20pt 1em;

```
#t1{  
border-collapse: separate;  
border-spacing: 45px;  
}
```

First_Name	Last_Name	Subject	Marks
James	Gosling	Maths	92
Alan	Rickman	Maths	89
Sam	Mendes	Maths	82

The border-spacing Property

border-spacing: 45px;

```
#t2{  
border-collapse: separate;  
border-spacing: 20pt 1em;  
}
```

First_Name	Last_Name	Subject	Marks
James	Gosling	Maths	92
Alan	Rickman	Maths	89
Sam	Mendes	Maths	82

CSS Border cont'd



- **border-image:** defines an image to be used as the element's
 - The border-image property can be applied to all elements except the elements of the internal table (such as tr, th, td) when border-collapse is set to collapse
- It is the shorthand property for border-image-source, border-image-slice, border-image-width, border-image-outset, and border-image-repeat
- Syntax
 - border-image: source slice width outset repeat | initial | inherit;

border-image: url('border.png')
60 / 20px 20px round;



border-image: url('diamond.png') 43 / 10px 15px
round stretch;



CSS Border cont'd



`border-image: url('diamond.png')
40 round stretch;`



`border-image: linear-gradient(orange,
yellow, green) 40 / 30px 10px stretch;`



`border-image: repeating-linear-gradient(50deg, blue,
yellow, lightgreen 20px) 30 / 20px 30px round;`



`border-image: radial-gradient(circle, yellow,
magenta, blue) 30 / 15px repeat;`



`border-image: radial-gradient(farthest-side,
red, yellow, green) 30 / 15px round;`



CSS Display



- **Display:** used to control the layout of the element
- **Syntax** display: inline | inline-block | block | run-in | none;

Property-value	Description
flex	It is used to display an element as an block-level flex container. It is new in css3.
inline-flex	It is used to display an element as an inline-level flex container. It is new in css3.
inline-table	It displays an element as an inline-level table.
list-Item	It makes the element behave like a element.
table	It makes the element behave like a <table> element.
table-caption	It makes the element behave like a <caption> element.
table-column-group	It makes the element behave like a <colgroup> element.
table-header-group	It makes the element behave like a <thead> element.
table-footer-group	It makes the element behave like a <tfoot> element.
table-row-group	It makes the element behave like a <tbody> element.
table-cell	It makes the element behave like a <td> element.
table-row	It makes the element behave like a <tr> element.
table-column	It makes the element behave like a <col> element.

CSS Cursor

e.g. `<div style = "cursor:help">help</div>`

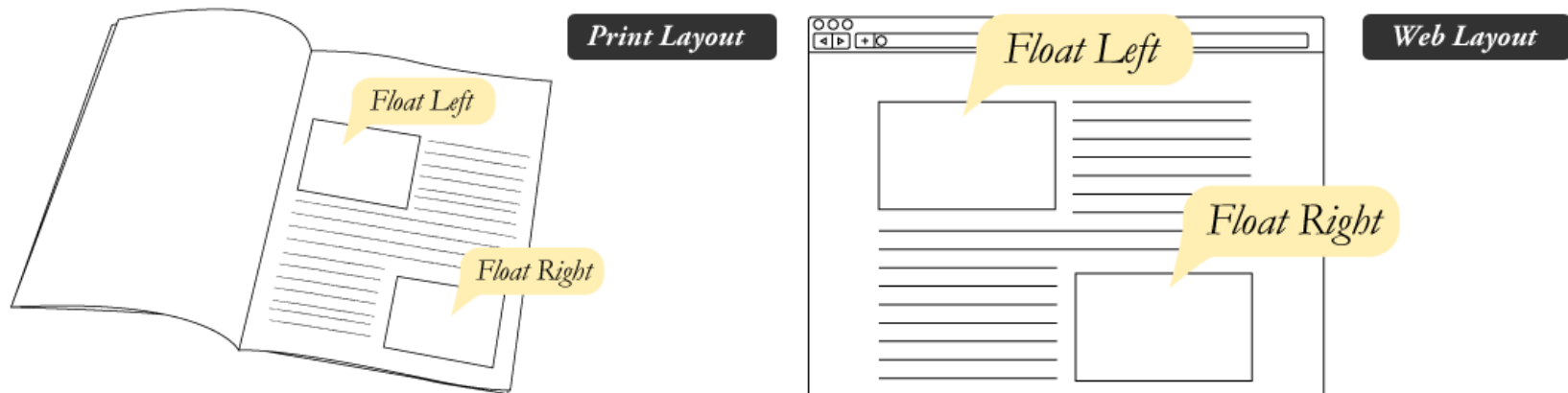
- It is used to define the type of mouse cursor when the mouse pointer is on the element



Values	Usage
alias	It is used to display the indication of the cursor of something that is to be created.
auto	It is the default property in which the browser sets the cursor.
all-scroll	It indicates the scrolling.
col-resize	Using it, the cursor will represent that the column can be horizontally resized.
cell	The cursor will represent that a cell or the collection of cells is selected.
context-menu	It indicates the availability of the context menu.
default	It indicates an arrow, which is the default cursor.
copy	It is used to indicate that something is copied.
crosshair	In it, the cursor changes to the crosshair or the plus sign.
e-resize	It represents the east direction and indicates that the edge of the box is to be shifted towards right.
ew-resize	It represents the east/west direction and indicates a bidirectional resize cursor.
n-resize	It represents the north direction that indicates that the edge of the box is to be shifted to up.
ne-resize	It represents the north/east direction and indicates that the edge of the box is to be shifted towards up and right.
move	It indicates that something is to be shifted.
help	It is in the form of a question mark or ballon, which represents that help is available.
None	It is used to indicate that no cursor is rendered for the element.
No-drop	It is used to represent that the dragged item cannot be dropped here.
s-resize	It indicates an edge box is to be moved down. It indicates the south direction.
Row-resize	It is used to indicate that the row can be vertically resized.
Se-resize	It represents the south/east direction, which indicates that an edge box is to be moved down and right.
Sw-resize	It represents south/west direction and indicates that an edge of the box is to be shifted towards down and left.
Wait	It represents an hourglass.
<url>	It indicates the source of the cursor image file.
w-resize	It indicates the west direction and represents that the edge of the box is to be shifted left.
Zoom-in	It is used to indicate that something can be zoomed in.
Zoom-out	It is used to indicate that something can be zoomed out.

CSS Float

- **Float:** is a positioning property used to push an element to the left or right, allowing other element to wrap around it
 - It is generally used with images and layouts



- Syntax
 - float: none | right | left | initial | inherit;

CSS Font

- Font property is used to control the look of texts



Font family: `p { font-family: monospace; }`

Font Size:

`<p style="font-size:xx-small;"> This font size is extremely small.</p>`

`<p style="font-size:x-small;"> This font size is extra small</p>`

`<p style="font-size:small;"> This font size is small</p>`

`<p style="font-size:medium;"> This font size is medium. </p>`

`<p style="font-size:large;"> This font size is large. </p>`

`<p style="font-size:x-large;"> This font size is extra large. </p>`

`<p style="font-size:xx-large;"> This font size is extra extra large. </p>`

`<p style="font-size:smaller;"> This font size is smaller</p>`

`<p style="font-size:larger;"> This font size is larger</p>`

`<p style="font-size:200%;"> This font size is 200% of the original</p>`

`<p style="font-size:20px;"> This font size is 20 pixels</p>`

Font Style:

`h2 { font-style: italic; }`

`h3 { font-style: oblique; }`

`h4 { font-style: normal; }`

Font Variant:

`p { font-variant: small-caps; }`

`h3 { font-variant: normal; }`

Font Weight:

`<p style="font-weight:bold;">This font is bold.</p>`

`<p style="font-weight:bolder;">This font is bolder.</p>`

`<p style="font-weight:lighter;">This font is lighter.</p>`

`<p style="font-weight:100;">This font is 100 weight.</p>`

`<p style="font-weight:200;">This font is 200 weight.</p>`

`<p style="font-weight:300;">This font is 300 weight.</p>`

`<p style="font-weight:400;">This font is 400 weight.</p>`

`<p style="font-weight:500;">This font is 500 weight.</p>`

`<p style="font-weight:600;">This font is 600 weight.</p>`

`<p style="font-weight:700;">This font is 700 weight.</p>`

`<p style="font-weight:800;">This font is 800 weight.</p>`

`<p style="font-weight:900;">This font is 900 weight.</p>`

CSS Font cont'd



- **font-stretch:** allows us to select a normal, expanded, or condensed face from the font's family
- Syntax
 - font-stretch: normal | semi-condensed | condensed | extra-condensed | ultra-condensed | semi-expanded | expanded | extra-expanded | ultra-expanded

Example of the font-stretch property

normal
semi-condensed
condensed
extra-condensed
ultra-condensed
semi-expanded
expanded
extra-expanded
ultra-expanded



- **Hover:** the :hover selector is for selecting the elements when we move the mouse on them
 - Syntax
 - :hover { css declarations; }
- **Important:** used to give more importance compare to normal property
 - Syntax

```
element {  
    font-size: 14px !important;  
    color: blue !important;    ...  
}
```
- **Line height:** used to define the minimal height of line boxes within the element

value	description
normal	This is a default value. it specifies a normal line height
number	It specifies a number that is multiplied with the current font size to set the line height
length	It is used to set the line height in px, pt,cm,etc
%	It specifies the line height in percent of the current font
initial	It sets this property to its default value
inherit	It inherits this property from its parent element

- **Opacity:** used to specify the transparency of an element (clarity of the image)



```
img.trans {  
    opacity: 0.4;  
    filter: alpha(opacity=40); /* For IE8 and earlier */  
}
```

- **Overflow:** specifies how to handle the content when it overflows its block level container
 - Syntax
 - overflow: visible | hidden | auto | inherit | initial | scroll;
- **Filter:** are used to set visual effects to text, images, and other aspects of a webpage
 - Syntax
 - filter: none | invert() | drop-shadow() | brightness() | saturate() | blur() | hue-rotate() | contrast() | opacity() | grayscale() | sepia() | url();

Filter example



sepia(90%)



grayscale(80%)



invert(60)



Original Image



hue-rotate(240deg)



opacity(40%)



blur(2px)



saturate(40)



contrast(50%)



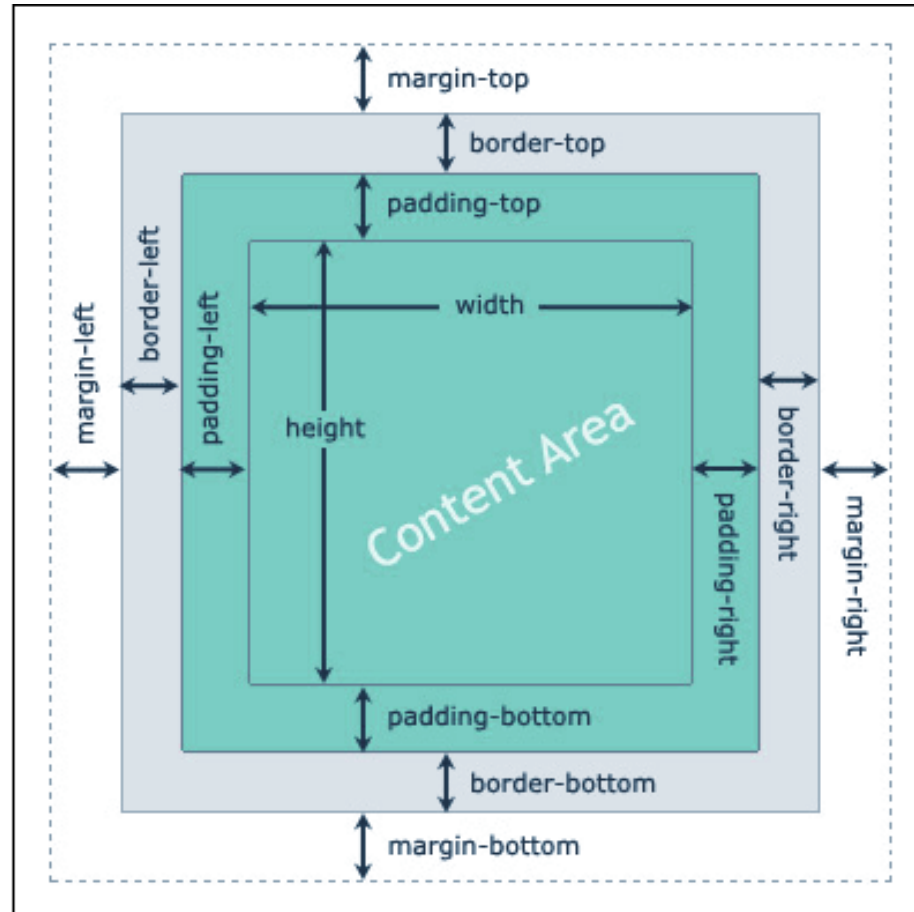
brightness(130%)



drop-shadow(10px 20px 30px yellow);

CSS Box Model

- The components that can be depicted on the web page consist of one or more than one rectangular box



- **Margin:** used to define the space around elements

Property	Description
margin	This property is used to set all the properties in one declaration.
margin-left	it is used to set left margin of an element.
margin-right	It is used to set right margin of an element.
margin-top	It is used to set top margin of an element.
margin-bottom	It is used to set bottom margin of an element.

Value	Description	Margin Values
auto	This is used to let the browser calculate a margin.	
length	It is used to specify a margin pt, px, cm, etc. its default value is 0px.	
%	It is used to define a margin in percent of the width of containing element.	
inherit	It is used to inherit margin from parent element.	

Example

```
.myBox {  
    margin-top: 50px;  
    margin-bottom: 50px;  
    margin-right: 100px;  
    margin-left: 100px;  
}
```

- **Padding:** used to define the space between the element content and the element border



Property	Description
padding	It is used to set all the padding properties in one declaration.
padding-left	It is used to set left padding of an element.
padding-right	It is used to set right padding of an element.
padding-top	It is used to set top padding of an element.
padding-bottom	It is used to set bottom padding of an element.

- **Padding Values**

Value	Description
length	It is used to define fixed padding in pt, px, em etc.
%	It defines padding in % of containing element.

Example

```
p.padding {  
  padding-top: 50px;  
  padding-right: 100px;  
  padding-bottom: 150px;  
  padding-left: 200px;  
}
```

CSS Width and Height

- **Width** property is used to set the width of the content area of an element
- **Height** property sets the height of an element
- Syntax
 - width: auto | length | initial | inherit;
 - height: auto | length | initial | inherit;



Example

```
#auto{  
    height: auto;  
    width: 275px;  
    border: 2px solid blue;  
}
```

Value	Description
auto	It is a default value. Using this value browser is responsible for calculating the width/height of the element. Negative values are not allowed.
length	It specifies the width/height of an element using the length units such as px, cm, pt, etc. Negative values are not allowed.
%	It defines the width/height of the container in %. Negative values are not allowed.
initial	It is used to set the property to its default value.
inherit	It is used to inherit the property from its parent element.

CSS Position



- **position:** is used to set position for an element
 - it is also used to place an element behind another and also useful for scripted animation effect
- fixed positioning property helps to put the text fixed on the browser

CSS Position example

- **fixed** positioning property helps to put the text fixed on the browser

```
p.pos_fixed { position: fixed;
top: 50px;
right: 5px;
color: blue; }
```

- **relative** positioning property is used to set the element relative to its normal position

```
h2.pos_right { position: relative;
left: 30px; }
```

- **absolute** positioning is used to position an element relative to the first parent element that has a position other than static

```
h2 { position: absolute;
left: 150px;
top: 250px; }
```

- **sticky** property is used to set the position for an element



- It is also used to place an item behind another element and also useful for the scripted animation effect
- The "position: sticky;" is used to position the element based on the scroll position of the user

```
<style>
body{ text-align:center; }
.stick{
position: sticky;
top:50px;
padding: 10px;
font-size:20px;
font-weight:bold;
background-color: lightblue;
border: 1px solid blue;
}
</style>
```

CSS Position cont'd

property	description	values
bottom	It is used to set the bottom margin edge for a positioned box.	auto, length, %, inherit
clip	It is used to clip an absolutely positioned element.	shape, auto, inherit
cursor	It is used to specify the type of cursors to be displayed.	url, auto, crosshair, default, pointer, move, e-resize, ne-resize, nw-resize, n-resize, se-resize, sw-resize, s-resize, w-resize, text, wait, help
left	It sets a left margin edge for a positioned box.	auto, length, %, inherit
overflow	This property is used to define what happens if content overflow an element's box.	auto, hidden, scroll, visible, inherit
position	It is used to specify the type of positioning for an element.	absolute, fixed, relative, static, inherit
right	It is used to set a right margin edge for a positioned box.	auto, length, %, inherit
top	It is used to set a top margin edge for a positioned box.	auto, length, %, inherit
z-index	It is used to set stack order of an element.	number, auto, inherit

CSS Vertical Align

- **vertical align:** is used to define the vertical alignment of an inline or table-cell box



value	description
baseline	It aligns the baseline of element with the baseline of parent element. This is a default value.
length	It is used to increase or decrease length of the element by the specified length. negative values are also allowed.
%	It is used to increase or decrease the element in a percent of the "line-height" property. negative values are allowed.
sub	It aligns the element as if it was subscript.
super	It aligns the element as if it was superscript.
top	It aligns the top of the element with the top of the tallest element on the line.
bottom	It aligns the bottom of the element with the lowest element on the line.
text-top	the top of the element is aligned with the top of the parent element's font.
middle	the element is placed in the middle of the parent element.
text-bottom	the bottom of the element is aligned with the bottom of the parent element's font.
initial	It sets this property to its default value.
inherit	inherits this property from its parent element.

CSS Vertical Align example



```
<style>
```

```
img.top {
```

```
    vertical-align: text-top;
```

```
}
```

```
img.bottom {
```

```
    vertical-align: text-bottom;
```

```
}
```

```
</style>
```

```
</head>
```

```
<body>
```

```
<p><img
```

```
  morning.jpg" alt="Good Morning Friends"/> This is an image  
with a default alignment.</p>
```

```
<p><img
```

```
  morning.jpg" class="top" alt="Good Morning Friends"/> This  
is an image with a text-top alignment.</p>
```

```
<p><img
```

```
  morning.jpg" class="bottom" alt="Good Morning Friends"/>  
This is an image with a text-bottom alignment.</p>
```

```
</body>
```



This is an image with a default alignment.



This is an image with a text-top alignment.



This is an image with a text-bottom alignment.

CSS White Space

CSS white space property is used to specify how to display the content within an element



Value	Description
normal	This is a default value. in this value, text is wrapped when necessary. sequences of white space will collapse into a single whitespace.
nowrap	Sequences of white space will collapse into a single whitespace. in this value, text will never wrap to the next line and only break when tag is used.
pre	Whitespace is preserved by the browser. it is act like html <pre> tag. text will only wrap on line breaks.
pre-line	Sequences of white space will collapse into a single whitespace. texts are wrapped when necessary, and on line break.
pre-wrap	Whitespace is preserved by the browser. texts are wrapped when necessary, and on line break.
initial	It sets this property to its default value.
inherit	It inherits this property from its parent element.

CSS Word Wrap

- **word wrap** property is used to break the long words and wrap onto the next line



Value	Description
normal	This property is used to break words only at allowed break points.
break-word	It is used to break unbreakable words.
initial	It is used to set this property to its default value.
inherit	It inherits this property from its parent element.

```
<style>  
p.test {  
    width: 11em;  
    background-color: #000000;  
    border: 1px solid #000000;  
    padding: 10px;  
    word-wrap: break-word;  
}  
</style>  
</head>  
<body>  
<p class="test"> In this paragraph  
iamsoooooooooooooooooooooo  
o the next line.</p>  
</body>
```

Output

In this paragraph, there is a very long word:
iamsooooooooooooooooooooo
oooooooooooooooooolongggggg
ggggggggggg. The long
word will break and wrap
to the next line.

break and wrap t

CSS Box-shadow



- It is used to add shadow-like effects around the frame of an element
- Syntax
 - `box-shadow: h-offset v-offset blur spread color |inset|inherit|initial|none;`
- `h-offset`: It horizontally sets the shadow position. Its positive value will set the shadow to the right side of the box. Its negative value is used to set the shadow on the left side of the box.
- `v-offset`: Unlike the `h-offset`, it is used to set the shadow position vertically. The positive value in it sets the shadow below the box, and the negative value sets the shadow above of the box.
- `blur`: As its name implies, it is used to blur the box-shadow. This attribute is optional.
- `spread`: It sets the shadow size. The spread size depends upon the spread value.
- `color`: As its name implies, this attribute is used to set the color of the shadow. It is an optional attribute.
- `inset`: Normally, the shadow generates outside of the box, but by using `inset`, the shadow can be created within the box.
- `initial`: It is used to set the property of the box-shadow to its default value.
- `inherit`: it is inherited from its parent.
- `none`: It is the default value that does not include any shadow property.

CSS Text-shadow



- **Text-shadow** property adds shadows to the text
- It accepts the comma-separated list of shadows that applied to the text
- Syntax
 - text-shadow: h-shadow v-shadow blur-radius color| none | initial | inherit;
- h-shadow: It is the required value. It specifies the position of the horizontal shadow and allows negative values.
- v-shadow: It is also the required value that specifies the position of the vertical shadow. It does not allow negative values.
- blur-radius: It is the blur-radius, which is an optional value. Its default value is 0.
- color: It is the color of the shadow and also an optional value.
- none: It is the default value, which means no shadow.
- initial: It is used to set the property to its default value.
- inherit: It simply inherits the property from its parent element.

CSS Box-shadow



```
div {  
border: 1px solid;  
padding: 10px; }  
#hvb {  
    /* box-shadow: h-offset v-offset blur */  
    box-shadow: 5px 10px 10px; }  
#spr { /* box-shadow: h-offset v-offset blur spread */  
    box-shadow: 5px 10px 10px 10px; }  
#col { /* box-shadow: h-offset v-offset blur spread color */  
    box-shadow: 5px 10px 10px 10px orange; }  
#ins { /* box-shadow: h-offset v-offset blur spread color inset */  
    box-shadow: 5px 10px 10px 10px orange inset; }  
#init { /* box-shadow: initial */  
    box-shadow: initial; }  
#non { /* box-shadow: none */  
    box-shadow: none; }
```

```
... ..  
<div id = "hvb">  
    <h1>It is a shadow box that has h-offset, v-offset and blur attributes.</h1>  
</div>  
<div id = "spr">  
    <h1>It is a box that includes the spread attribute.</h1>  
</div>  
<div id = "col">  
    <h1>It is a box that includes the color attribute.</h1>  
</div>  
<div id = "ins">  
    <h1>It is a box that includes the inset attribute.</h1>  
</div>  
<div id = "init">  
    <h1>It is a box that includes the initial attribute.</h1>  
</div>  
<div id = "non">  
    <h1>It is a box that includes the default attribute i.e. none.</h1>  
</div>
```

It is a shadow box that has h-offset, v-offset and blur attributes.

It is a box that includes the spread attribute.

It is a box that includes the color attribute.

It is a box that includes the inset attribute.

It is a box that includes the initial attribute.

It is a box that includes the default attribute i.e. none.

CSS Text-shadow example

```

<style>
  p.simple{
    text-shadow: 3px 3px red;
  }
</style>
.....
<p class="simple">
  Simple Shadow
</p>
  
```

Output

Simple Shadow

`text-shadow: 3px 3px 3px violet;`

Fuzzy Shadow

`text-shadow: 0 0 .2em yellow;`

Glow Effect

`text-shadow: -3px -3px 3px blue, 3px 3px 3px red;`

Multiple Shadows

- **Text-transform** property allows us to change the case of the text



- Syntax
 - text-transform: capitalize | uppercase | lowercase | none | initial | inherit;
- **Outline** is just like CSS border property
 - Facilitates to draw an extra border around an element to get visual attention
 - Difference between Border and Outline
 - It is not possible to apply a different outline width, style and color for the four sides of an element while in border;
 - The border is a part of element's dimension while the outline is not the part of element's dimension

CSS outline example



```
<!DOCTYPE html>
<html>
<style type="text/css">
.box {
    background-color: #eee;
    border: 3px solid lightgreen;
    padding: 5px 10px;
    outline-width: 5px;
    outline-style: solid;
    outline-color: red;
}
</style>
<div class="box">Hello World</div>
</body>
</html>
```

Hello World

```
<!DOCTYPE html>
<html>
<style type="text/css">
.box {
    background-color: #eee;
    outline: 3px solid red;
    outline-offset: 6px;
    border: 3px solid lightgreen;
    padding: 5px 10px;
}
</style>
<div class="box">Welcome to
JavaTpoint</div>
</body>
</html>
```

Welcome to JavaTpoint

```
border: 2px solid lightgreen;
padding: 5px 10px;
```

```
}
</style>
<div class="box" style="outline-style: dashed;">This is dashed outline.</div>
<div class="box" style="outline-style: dotted;">This is dotted outline.</div>
<div class="box" style="outline-style: double;">This is double outline.</div>
<div class="box" style="outline-style: groove;">This is groove outline.</div>
<div class="box" style="outline-style: inset;">This is inset outline.</div>
<div class="box" style="outline-style: outset;">This is outset outline.</div>
<div class="box" style="outline-style: ridge;">This is ridge outline.</div>
<div class="box" style="outline-style: solid;">This is solid outline.</div>
```

This is dashed outline.

This is dotted outline.

This is double outline.

This is groove outline.

This is inset outline.

This is outset outline.

This is ridge outline.

This is solid outline.



Cascading Style Sheet (CSS)

Internet Programming I: Chapter 3 – Part II



ADDIS ABABA SCIENCE AND TECHNOLOGY UNIVERSITY
Department of Software Engineering

CSS counters

- **counters** are similar to variables, maintained by CSS and those values can be incremented by CSS rules to track how many times they are used
- CSS Counter Properties
 - **counter-reset**: It is used to create or reset a counter
 - **counter-increment**: It is used to increment the counter value
 - **content**: It is used to insert generated content
 - **counter()** or **counters()** function: It is used to add the value of a counter to an element



```
body {  
    counter-reset: section;  
}  
h2::before {  
    counter-increment: section;  
    content: "Section " counter(section) " " ;  
}  
... ..  
<h1>Example of CSS Counters:</h1>  
<h2>Java Tutorial</h2>  
<h2>HTML Tutorial</h2>  
<h2>CSS Tutorial</h2>  
<h2>Oracle Tutorial</h2>  
<p><strong>Note:</strong> IE8 supports these properties only if a !DOCTYPE is specified.</p>
```

Example of CSS Counters:

Section 1: Java Tutorial

Section 2: HTML Tutorial

Section 3: CSS Tutorial

Section 4: Oracle Tutorial

Note: IE8 supports these properties only if a !DOCTYPE is specified.

Nesting Counters example



```
body { counter-reset: section; }
h1 { counter-reset: subsection; }
h1::before {
  counter-increment: section;
  content: "Section " counter(section) ". "; }
h2::before {
  counter-increment: subsection;
  content: counter(section) "." counter(subsection) " "; }
```

... ..

```
<h1>Java tutorials:</h1>
<h2>Core Java tutorial</h2>
<h2>Servlet tutorial</h2>
<h2>JSP tutorial</h2>
<h2>Spring tutorial</h2>
<h2>Hibernate tutorial</h2>
```

```
<h1>Web technology tutorials:</h1>
<h2>HTML tutorial</h2>
<h2>CSS tutorial</h2>
<h2>jQuery tutorial</h2>
<h2>Bootstrap tutorial</h2>
```

```
<h1>Database tutorials:</h1>
<h2>SQL tutorial</h2>
<h2>MySQL tutorial</h2>
<h2>PL/SQL tutorial</h2>
<h2>Oracle tutorial</h2>
```

Note: IE8 supports these properties only if a !DOCTYPE is specified.

Section 1. Java tutorials:

1.1 Core Java tutorial

1.2 Servlet tutorial

1.3 JSP tutorial

1.4 Spring tutorial

1.5 Hibernate tutorial

Section 2. Web technology tutorials:

2.1 HTML tutorial

2.2 CSS tutorial

2.3 jQuery tutorial

2.4 Bootstrap tutorial

Section 3. Database tutorials:

3.1 SQL tutorial

3.2 MySQL tutorial

3.3 PL/SQL tutorial

3.4 Oracle tutorial

Note: IE8 supports these properties only if a !DOCTYPE is specified.

```

ol { counter-reset: section;
    list-style-type: none; }
li::before { counter-increment: section;
    content: counters(section, ".") " "; }

```

... ..

<h2>Different level of Nesting counters</h2>

item

item

item

item

item

item

item

item

item

item

item

<p>Note: IE8 supports these properties only if a !DOCTYPE is specified.</p>

Different level of nesting counters example

Different level of Nesting counters

```

1 item
2 item
    2.1 item
    2.2 item
    2.3 item
        2.3.1 item
        2.3.2 item
        2.3.3 item
    2.4 item
3 item
4 item

```

Note: IE8 supports these properties only if a !DOCTYPE is specified.

```
div {
  border: 3px solid red;
  padding: 5px;
  background-color: pink;
  font-size: 20px;
}
```

```
img { float: right;
  border: 3px solid blue;
}
p { font-size: 20px;
  clear: right; }
.jtp { overflow: auto; }
```

..... ..

<h1>Without Clearfix</h1>

<div>

Welcome to the javaTpoint.com. Here, you can find the best tutorials that are easy to read and help you to clear your concepts.

</div>

<p>Use the overflow:auto; CSS property</p>

<h1>With Clearfix</h1>

<div class="jtp">

Welcome to the javaTpoint.com. Here, you can find the best tutorials that are easy to read and help you to clear your concepts.

</div>

- A **clear float (or clearfix)** is a way for an element to fix or clear the child elements so that we do not require to add additional markup
- It resolves the error which occurs when more than one floated elements are stacked next to each other

Without Clearfix

Welcome to the javaTpoint.com. Here, you can find the best tutorials that are easy to read and help you to clear your concepts.



Use the overflow:auto; CSS property

With Clearfix

Welcome to the javaTpoint.com. Here, you can find the best tutorials that are easy to read and help you to clear your concepts.



....

```
div { border: 3px solid red;
padding: 5px;
background-color: pink;
font-size: 20px; }
img { float: right;
border: 3px solid blue; }
p { font-size: 20px;
clear: right; }
.jtp:after { content: '.';
clear: both;
display: table; }
```

....

<h1>Without Clearfix</h1>

<div>

Welcome to the javaTpoint.com. Here, you can find the best tutorials that are easy to read and help you to clear your concepts.

</div>

<p>Another clearfix method</p>

<h1>With Clearfix</h1>

<div class="jtp">

Welcome to the javaTpoint.com. Here, you can find the best tutorials that are easy to read and help you to clear your concepts.

</div>

Another clearfix method

- In this example, we set the clear property to 'both', which stands that the floating elements will not allow on both the left and right sides
- We set the display property to 'table', which makes the element behave like <table> element of HTML. And we also have to leave the content empty

Without Clearfix

Welcome to the javaTpoint.com. Here, you can find the best tutorials that are easy to read and help you to clear your concepts.



Another clearfix method

With Clearfix

Welcome to the javaTpoint.com. Here, you can find the best tutorials that are easy to read and help you to clear your concepts.



CSS icons



- Icons can be defined as the images or symbols used in any computer interface refer to an element
- Using the icon library is the easiest way to add icons to our HTML page
 - Eg. Bootstrap icons, Font Awesome icons, and Google icons
- It is possible to format the library icons by using CSS
- There is no need to install or download the libraries mentioned

```
<link rel="stylesheet" href="https://cdnjs.cloudflare.com/ajax/libs/font-awesome/4.7.0/css/font-awesome.min.css">
```

```
<style>
```

```
body{ text-align:center;  
background-color:lightblue; }
```

```
.fa{ color:red;  
font-size:50px;  
margin:10px; }
```

```
.....
```

```
<h1>Font Awesome Library</h1>  
<i class="fa fa-cloud"></i>  
<i class="fa fa-file"></i>  
<i class="fa fa-heart"></i>  
<i class="fa fa-bars"></i>  
<i class="fa fa-car"></i>
```

Example: using font Awesome icons



```
<link rel="stylesheet" href="https://maxcdn.bootstrapcdn.com/bootstrap/3.3.7/css/bootstrap.min.css">
```

```
<style>
```

```
body{ text-align:center;
background-color:lightblue; }
```

```
.glyphicon {
color:red;
font-size:50px;
margin:10px; }
```

```
</style>
```

```
</head>
```

```
<h1>Bootstrap icons</h1>
```

```
<i class="glyphicon glyphicon-cloud"></i>
```

```
<i class="glyphicon glyphicon-file"></i>
```

```
<i class="glyphicon glyphicon-heart"></i>
```

```
<i class="glyphicon glyphicon-user"></i>
```

```
<i class="glyphicon glyphicon-thumbs-up"></i>
```

```
<i class="glyphicon glyphicon-remove"></i>
```

```
<i class="glyphicon glyphicon-envelope"></i>
```

Example: using Bootstrap icons



```
.....
```

```
<link rel="stylesheet" href="https://fonts.googleapis.com/icon?family=Material+Icons">
```

```
<style>
```

```
body{ text-align:center;
background-color:lightblue; }
```

```
.material-icons{ color:red;
font-size:50px;
margin:10px; }
```

```
.....
```

```
<body>
```

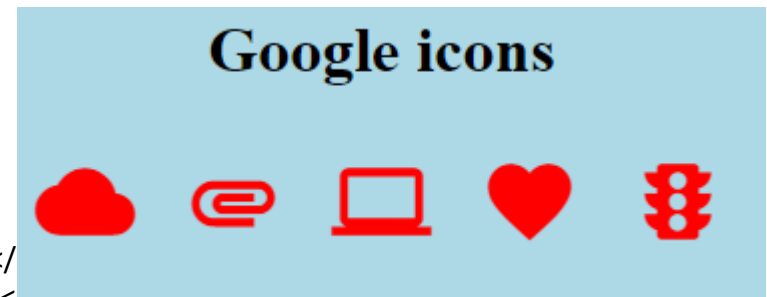
```
<h1>Google icons</h1>
```

```
<i class="material-icons">cloud</i> <i class="material-icons">attachment</i>
```

```
<i class="material-icons">computer</i> <i class="material-icons">favorite</i>
```

```
<i class="material-icons">traffic</i>
```

Example: Google icons



justify-content

- **Justify-content** property is used to align the items of the flexible box container when the items do not use all available space on the main-axis (horizontally)
- It defines how the browser distributes the space around and between the content items
- Syntax
 - justify-content: center | flex-start | flex-end | space-around | space-evenly | space-between | initial | inherit;
- The default value of this property is flex-start
- **center**: As its name implies, it set the position of items at the center of the container
- **flex-start**: It is the default value that positioned the items at the beginning of the container
- **flex-end**: It set the position of items at the end of the container.
- **space-around**: It positioned the items with equal spacing from each other. It evenly distributes the items in the line along with the same space around them
- **space-between**: Items are evenly spaced in which the first item is at the beginning, and the last item is at the end
- **space-evenly**: It also positioned the items with equal space, but the spacing from the corners differs

justify-content example



flex-start 1 2 3 4 5

flex-end 1 2 3 4 5

center 1 2 3 4 5

space-evenly

1 2 3 4 5

space-around

1 2 3 4 5

space-between

1 2 3 4 5

CSS Lists

- The CSS properties to style the lists are given as follows:
 - **list-style-type**: This property is responsible for controlling the appearance and shape of the marker
 - **list-style-image**: It sets an image for the marker instead of the number or a bullet point
 - **list-style-position**: It specifies the position of the marker.
 - **list-style**: It is the shorthand property of the above properties
 - **marker-offset**: It is used to specify the distance between the text and the marker. It is unsupported in IE6 or Netscape 7



- **list-style-type**:

```
.....  
<style>  
  .num { list-style-type:decimal; }  
  .alpha{ list-style-type:lower-alpha; }  
  .roman{ list-style-type:lower-roman; }  
  .circle{ list-style-type:circle; }  
  .square{ list-style-type:square; }  
  .disc{ list-style-type:disc; }  
.....
```

```
<h2> Ordered Lists </h2>  
<ol class="num">  
  <li>one</li>  
  <li>two</li>  
  <li>three</li>  
</ol>  
<ol class="alpha">  
  <li>one</li>  
  <li>two</li>  
  <li>three</li>  
</ol>  
  <ol class="roman">  
    <li>one</li>  
    <li>two</li>  
    <li>three</li>  
  </ol>
```

```
<h2> Unordered lists </h2>  
<ul class="disc">  
  <li>one</li>  
  <li>two</li>  
  <li>three</li>  
</ul>  
<ul class="circle">  
  <li>one</li>  
  <li>two</li>  
  <li>three</li>  
</ul>  
<ul class="square">  
  <li>one</li>  
  <li>two</li>  
  <li>three</li>  
</ul>
```

CSS Lists cont'd

- **list-style-position** It represents whether the appearing of the marker is inside or outside of the box containing the bullet points

- **inside:** It means that the bullet points will be in the list item
 - In this, if the text goes on the second line, then the text will be wrap under the marker
- **outside:** It represents that the bullet points will be outside the list item. It is the default value

```
<style>
  .num{ list-style-type:decimal;
        list-style-position:inside; }
  .roman{ list-style-type:lower-roman;
          list-style-position:outside; }
  .circle{ list-style-type:circle;
           list-style-position:inside; }
  .square{ list-style-type:square; }
  .disc{ list-style-type:disc;
         list-style-position:inside; }
</style>
```

Ordered Lists

1. INSIDE
2. two
3. three

- i. OUTSIDE
- ii. two
- iii. three

Unordered lists

- INSIDE
- two
- three

- INSIDE
- two
- three

- DEFAULT
- two
- three

list-style-image



- It specifies an image as the marker
- Its syntax is similar to the background-image property. If it does not find the corresponding image, the default bullets will be used

```
<style>
  .order{
    list-style-image: url(img.png);
  }
  .unordered{
    list-style-image: url(img.png);
  }
</style>
```

Ordered Lists

- 
- one
-
- 
- two
-
- 
- three

Unordered lists

- 
- one
-
- 
- two
-
- 
- three

list-style

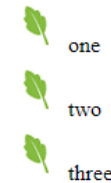
- **list-style** is the shorthand property that is used to set all list properties in one expression
 - The order of the values of this property is type, position, and image. But if any property value is missing, then the default value will be inserted



```
<style>
.order{ list-style: lower-alpha inside url(img.png); }
.unorder{ list-style: disc outside; }
```

```
.order{ list-style: upper-alpha;
        background: pink;
        padding:20px;  }
.order li{ background: lightblue;
padding:10px;
font-size:20px;
margin:10px;  }
.unorder{ list-style: square inside;
        background: cyan;
        padding:20px;  }
.unorder li{ background: green;
color:white;
padding:10px;
font-size:20px;
margin:10px;  }
```

Ordered Lists



Unordered lists

- one
- two
- three

- A. ONE
B. TWO
C. THREE

Unordered lists

- ONE
- TWO
- THREE

CSS :nth-child(n) selector

- **:nth-child(n)** selector is used for matching the nth-child elements based on their position regardless of the type of its parent
 - The n can either be a keyword, formula, or a number
 - It is used to match the elements based on their position within a group of siblings
- Syntax
 - :nth-child(n) { //CSS Property }
- **Functional notation (An+B):** Functional notation represents those elements whose siblings position match **the An+B** pattern
 - where **A** is the integer step size, **n** is any positive integer starting from 0, and **B** is the integer offset



```
p:nth-child(3n+4) { background: yellow;
                    font-size:30px; }
tr:nth-child(2n) { background: green; }
```

.....

```
<body style = "text-align:center">
  <h1> Hello World </h1>
  <h2> Welcome to the javaTpoint </h2>
  <p>It will not affected.</p>
  <p>It will be affected.</p>
  <p>Not affected.</p>
  <p>Not affected.</p>
  <p>It will be affected.</p>
  <table border=1>
    <tr> <td>1</td> <td>2</td> <td>3</td></tr>
    <tr> .....
```

Hello World

Welcome to the world

It will not affected.

It will be affected.

Not affected.

Not affected.

It will be affected.

1	2	3
21	22	23
31	32	33
1	2	3
21	22	23
31	32	33

text-decoration



- **text-decoration** property that decorates the content of the text
 - This is shorthand for text-decoration-line, text-decoration-color, and text-decoration-style
- Syntax
 - text-decoration: text-decoration-line text-decoration-color text-decoration-style | initial | inherit;

```
.....
#p1 { text-decoration: underline double cyan; }
#p2 { text-decoration: line-through dashed green; }
#p3 { text-decoration: dotted overline blue; }
#p4 { text-decoration: lightblue wavy overline underline line-through;
      color:red; }
</style>
....
<body>
<h2>text-decoration: text-decoration-line text-decoration-style;</h2>
<div>
  <p id="p1">This is double underline</p>
  <p id="p2">This is dashed line-through</p>
  <p id="p3">This is dotted overline</p>
  <p id="p4">This is the wavy combination of lines</p>
</div>
...
```

text-decoration: text-decoration-line text-decoration-style;

This is double underline

~~This is dashed line-through~~

.....
This is dotted overline

This is the wavy combination of lines

CSS letter-spacing



- The letter-spacing property used to control the space between every letter inside an element or the block of text
- Syntax
 - letter-spacing: normal | length | initial | inherit;

This is an example of CSS letter-spacing Property

This paragraph has letter-spacing: normal;

T h i s p a r a g r a p h h a s l e t t e r - s p a c i n g : 7 p x ;

a p h h a s l e t t e r - s p a c i n g :
0 . 7 e m ;

This paragraph has letter-spacing: -1px;

```
<p style= "letter-spacing: normal;">
```

This paragraph has letter-spacing: normal;

```
</p>
```

```
<p style= "letter-spacing: 7px;">
```

This paragraph has letter-spacing: 7px; </p>

```
<p style= "letter-spacing: 0.7em;">
```

This paragraph has letter-spacing: 0.7em; </p>

```
<p style= "letter-spacing: -1px;">
```

This paragraph has letter-spacing: -1px; </p>

CSS text-indent

- This CSS property sets the indentation of the first line in a block of text
- Syntax
 - **text-indent:** length | inherit | initial;



Example of text-indent Property

text-indent: 70px;

Hi, Welcome to the javaTpoint.com. This site is developed so that students may learn computer science related technologies easily. The javaTpoint.com is always providing an easy and in-depth tutorial on various technologies. No one is perfect in this world, and nothing is eternally best. But we can try to be better.

text-indent: -5em;

Hi, Welcome to the javaTpoint.com. This site is developed so that students may learn computer science related technologies easily. The javaTpoint.com is always providing an easy and in-depth tutorial on various technologies. No one is perfect in this world, and nothing is eternally best. But we can try to be better.

text-indent: 7cm;

Hi, Welcome to the javaTpoint.com. This site is developed so that students may learn computer science related technologies easily. The javaTpoint.com is always providing an easy and in-depth tutorial on various technologies. No one is perfect in this world, and nothing is eternally best. But we can try to be better.

```
div{ font-size: 20px;
    width: 500px;
    height:200px;
    text-align: justify; }
.jtppx {text-indent: 100px; }
.jtpem { text-indent: -5em; }
.jtpcm { text-indent: 7cm; }
.....
<h1>Example of text-indent Property</h1>
<h2>text-indent: 70px;</h2>
<div class = "jtppx"> Hi, Wel... so that .... </div>
    <h2>text-indent: -5em;</h2>
<div class = "jtpem"> Hi, We ...computer .. </div>
    <h2>text-indent: 7cm;</h2>
<div class = "jtpcm"> Hi, Welcome ...learn </div>
```

CSS text-stroke



- This CSS property adds a stroke to the text and also provides decoration options for them
 - It is the shorthand of the **text-stroke-width** and **text-stroke-color**
- The text-stroke can only be used with the -webkit- prefix

```
.jtp {  
  color: yellow;  
  font-size: 50px;  
  -webkit-text-stroke-width: 2px;  
  -webkit-text-stroke-color: red;  
}
```

.....

```
<h1 class= "jtp">Welcome to the  
javaTpoint.com</h1>  
<h2 class= "jtp" style= "-webkit-text-  
stroke-color: blue;">This is an example  
of CSS text-stroke property</h2>  
</body>
```

Welcome to the
javaTpoint.com

This is an example of CSS
text-stroke property

CSS Text Effects

- **word-break:** specifies how words should break at the end of the line. It defines the line break rules
- Syntax
 - word-break: normal | keep-all | break-all | inherit ;
 - The default value of this property is normal
 - **keep-all:** It breaks the word in the default style
 - **break-all:** It inserts the word break between the characters in order to prevent the word overflow



```
.jtp{
  width: 150px;
  border: 2px solid black;
  word-break: break-all;
  text-align: left;
  font-size: 25px;
  color: blue;
}
```

```
.jtp1{
  width: 156px;
  border: 2px solid black;
  word-break: keep-all;
  text-align: left;
  font-size: 25px;
  color: blue;
}
```

```
.....
<h2>word-break: break-all;</h2>
<p class="jtp"> Welcome to the javaTpoint.com</p>
<h2>word-break: keep-all;</h2>
<p class="jtp1"> Welcome to the javaTpoint.com </p>
```

word-break: break-all;

Welcome to the
javaTpoint.com

word-break: keep-all;

Welcome to
the
javaTpoint.com

text-overflow

- text-overflow specifies the representation of overflowed text, which is not visible to the user
- This property does not work on its own. We have to use white-space: nowrap; and overflow: hidden; with this property
- Syntax
 - text-overflow: clip | ellipsis;
 - **clip**: It is the default value that clips the overflowed text
 - **ellipsis**: This value displays an ellipsis (...) or three dots to show the clipped text



```
.jtp{ white-space: nowrap;  
      height: 30px;  
      width: 250px;  
      overflow: hidden;  
      border: 2px solid black;  
      font-size: 25px;  
      text-overflow: clip; }
```

```
.jtp1 { white-space: nowrap;  
        height: 30px;  
        width: 250px;  
        overflow: hidden;  
        border: 2px solid black;  
        font-size: 25px;  
        text-overflow: ellipsis; }
```

.....

```
<p> Hover over the bordered text to see the full content. </p>
```

```
<h2> text-overflow: clip; </h2>
```

```
<div class="jtp"> Welcome to the javaTpoint.com </div>
```

```
<h2> text-overflow: ellipsis; </h2>
```

```
<div class="jtp1"> Welcome to the javaTpoint.com </div>....
```

Hover over the bordered text to see the full content.

text-overflow: clip;

Welcome to the javaTpoi

text-overflow: ellipsis;

Welcome to the javaT...

CSS writing-mode

- writing-mode: specifies whether the text will be written in the horizontal or vertical direction
- Syntax
 - writing-mode: horizontal-tb | vertical-lr | vertical-rl | inherit ;
 - **horizontal-tb**: It is the default value of this property in which the text is in the horizontal direction and read from left to right and top to bottom
 - **vertical-rl**: It displays the text in the vertical direction, and the text is read from right to left and top to bottom.
 - **vertical-lr**: It is similar to vertical-rl, but the text is read from left to right.

```
....
h2 { border: 2px solid black;
width: 300px;
height: 100px; }
#tb { writing-mode: horizontal-tb; }
#lr { writing-mode: vertical-lr; }
#rl { writing-mode: vertical-rl; }
....
<h1> Example of CSS writing-mode property </h1>
<h2 id="tb"> writing-mode: horizontal-tb; </h2>
<h2 id="lr" style= "height: 300px;"> writing-mode:
vertical-lr; </h2><br>
<h2 id="rl" style= "height: 300px;"> writing-mode:
vertical-rl; </h2>.....
```

writing-mode: horizontal-tb;

writing-mode: vertical-lr;

writing-mode: vertical-rl;

CSS text-align

- **text-align** property is used to set the horizontal alignment of a table-cell box or the block element
- Syntax
 - text-align: justify | center | right | left | initial | inherit;
 - justify: It is generally used in newspapers and magazines. It stretches the element's content in order to display the equal width of every line.
 - center: It centers the inline text.
 - right: It is used to align the text to the right
 - left: It is used to align the text to the left



```
....  
<h1>Example of text-align proeprty</h1>  
<h2 style = "text-align: center;"> text-align: center; </h2>  
  
<h2 style = "text-align: right;"> text-align: right;</h2>  
<h2 style = "text-align: left;">text-align: left; </h2>  
<h2 style = "text-align: justify;"> text-align: justify; To see  
its effect, it should be applied on large paragraph </h2>
```

text-align: center;

text-align: right;

text-align: left;

text-align: justify; To see its effect, it should be applied on large paragraph.

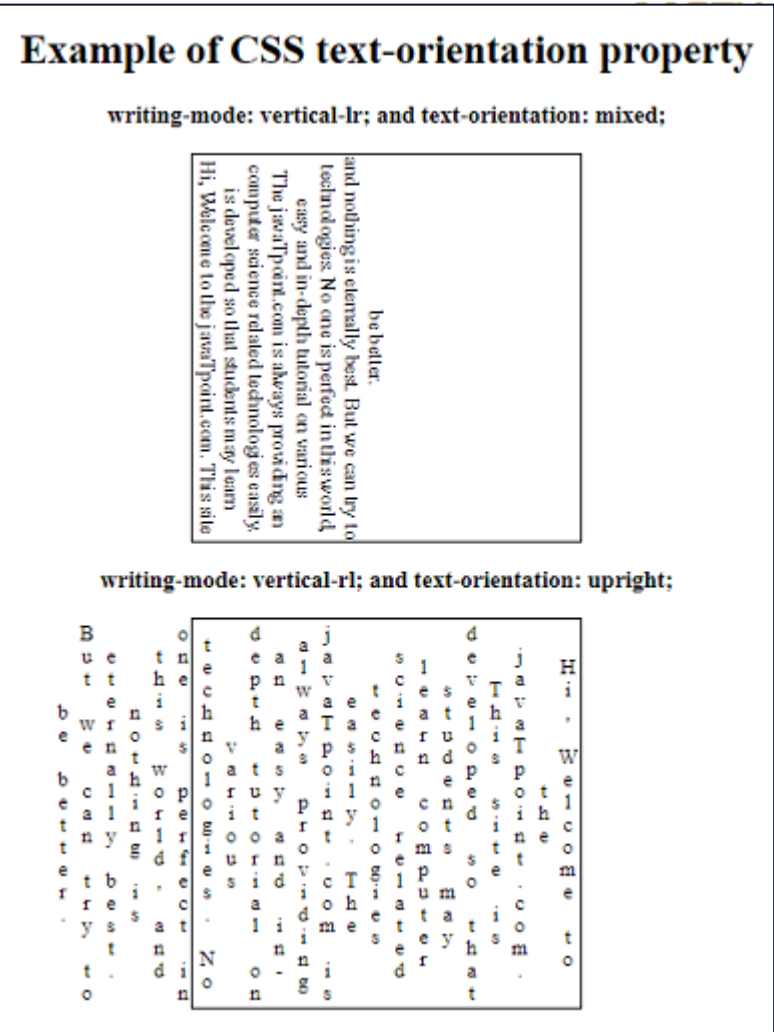
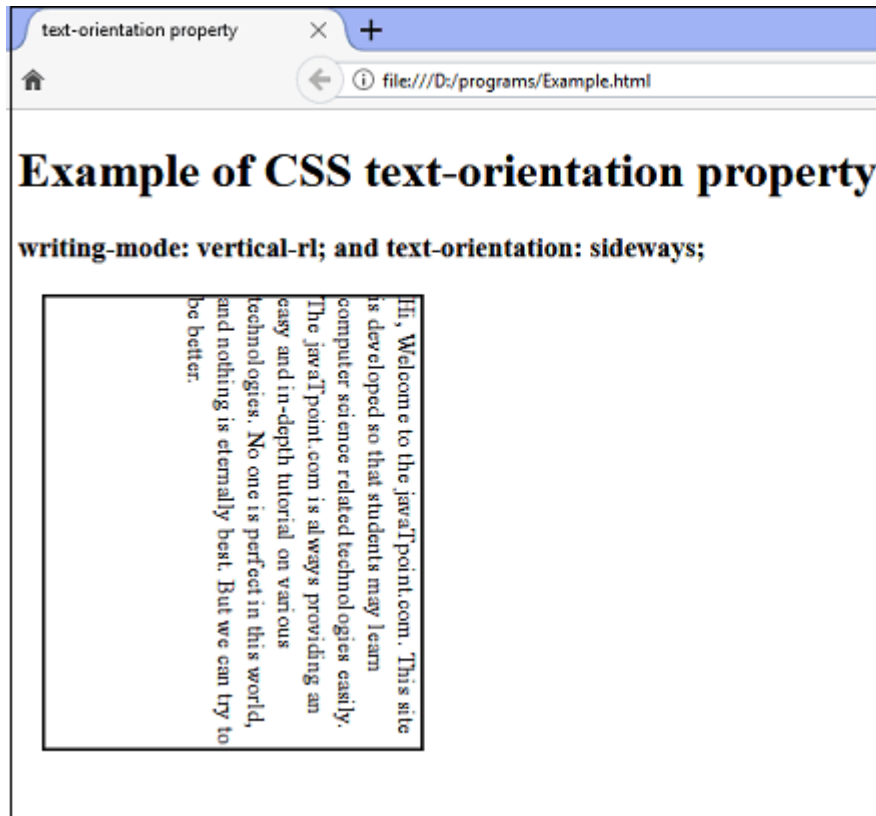
CSS text-orientation

- **text-orientation** property specifies the orientation of characters in the line of content
 - It only applies to the vertical mode of content. This property does not affect elements with horizontal writing mode
- Syntax
 - text-orientation: mixed | upright | sideways | sideways-right | use-glyph-orientation | initial | inherit;



Values	Description
mixed	It is the default value that rotates the characters 90o degree clockwise. It set the characters of vertical script naturally.
upright	This value sets the characters of horizontal scripts naturally (upright), as well as the glyphs for the vertical scripts. It makes all characters to be considered as left-to-right.
sideways	It rotates the line to 90o clockwise. This value causes the characters to be laid out as horizontally. This value does not work in Google Chrome and other major browsers except Firefox, i.e., it only works in Firefox.
sideways-right	It is kept for compatibility purposes. It is an alias to the value sideways.
use-glyph-orientation	This value is not used anymore.
initial	It sets the property to its default value.
inherit	It inherits the property from its parent element.

CSS text-orientation example



CSS :root Selector

- This pseudo-class in CSS matches the root element of the document
 - It selects the highest-level parent in the document tree or DOM
- In HTML, the <html> element is always the root element. Although the :root is equivalent to html selector because both of them target the same element, but the :root selector has a higher specificity

- Syntax

- :root { // CSS property }

```
...
:root {
    background: lightblue;
    color: blue;
    text-align: center;
}
</style>
</head>
<body>
    <h1>Welcome to the javaTpoint.com</h1>
    <h2>This is an example of :root selector</h2>
</body>
```

Welcome to the javaTpoint.com

This is an example of :root selector



CSS :root Selector example



In this example, we can see that the **background-color** and **color** properties are both mentioned in **html** as well as in **:root** selector, but in the output, the properties mentioned in **:root** selector will work. This is because of the higher specificity of the **:root** selector

```
.....  
<style>  
  :root { background-color: lightblue;  
          color: blue;  
          text-align: center; }  
  html { background-color: red;  
         color: white;  
         font-size: 30px; }  
</style>  
</head>  
<body>  
  <h1>Welcome to the javaTpoint.com</h1>  
  <h2>This is an example of :root selector and html  
selector</h2>  
.....
```

**Welcome to the
javaTpoint.com**

**This is an example of :root selector
and html selector**

CSS calc()



- It is an inbuilt CSS function which allows us to perform calculations
- It can be used to calculate length, percentage, time, number, integer frequency, or angle
- It uses the four simple arithmetic operators add (+), multiply (*), subtract (-), and divide (/)
- Allows us to mix any units, such as percentage and pixel
- Syntax: `calc( Expression );`
- Important points
 - The arithmetic operators add (+) and subtract (-) should always be surrounded by whitespace. Otherwise, the expression will be treated as an invalid expression
 - Example, `calc(60%-4px)` will be invalid because it is parsed as a percentage, followed by a negative length. On the other hand, the expression `calc(60% - 4px)` will be parsed as a subtraction operator and a length
 - Although the operators * and / does not requires whitespace, but it is recommended to add it for consistency
 - Nesting of `calc()` function can be possible

CSS calc() example

```
<style>
.jtp {
  width: calc(150% - 75%);
  height: calc(350px - 75px);
  background-color: lightblue;
  padding-top: 50px;
}
.jtp1 {
  font-size: 30px;
  font-weight: bold;
  color: blue;
}
....
</style>
```

Using mixed unit

```
<style>
.jtp {
  width: calc(40% + 10em);
  height: calc(350px + 75px);
  background-color: lightblue;
  padding-top: calc(10% - 10px);
  padding-left: calc(10% + 10px);
  font-size: 30px;
}
...
</style>
```



Nested calc()

```
<style>
.jtp {
  width: calc( calc(40em / 3) * 2);
  height: calc(350px + 75px);
  background-color: lightblue;
}
</style>
```

CSS Zoom

- This CSS property scales the content. It manages the magnification level of the content
 - Instead of using this property, we can also use the transform: scale(); property
- Syntax
 - zoom: normal | number | percentage | reset ;

```
....  
.magnify1{ zoom: 0.75; }  
.magnify{ zoom: 1.5; }  
</style>  
.....  
<body>  
...  
  
<h2>Magnified image zoom: 1.5;</h2>  

```

original image



Magnified image zoom: 0.75;

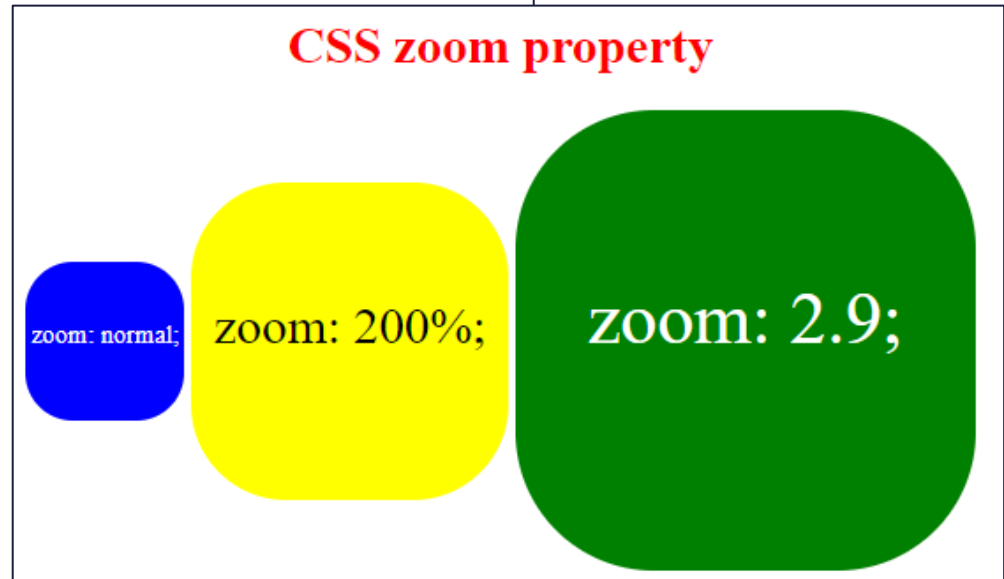


Magnified image zoom: 1.5;



CSS Zoom cont'd

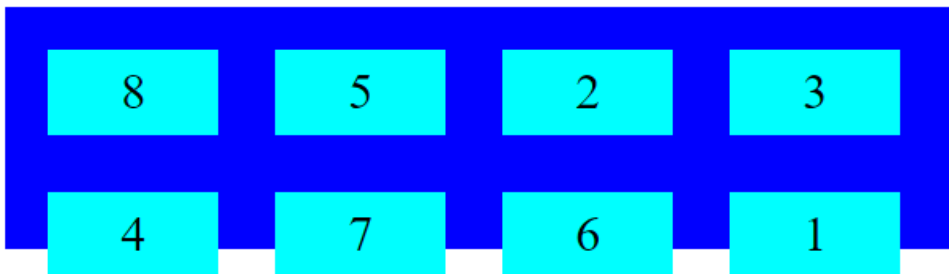
```
...  
.magnify{ width: 100px;  
  height: 100px;  
  border-radius: 30px;  
  display: inline-block;  
  color: white; }  
#m1 { background-color: blue;  
  zoom: normal; }  
#m2 { background-color: yellow;  
  zoom: 200%;  
  color: black; }  
#m3 { background-color: green;  
  zoom: 2.9; }  
p{ padding-top:20px; }  
</style>  
...  
<body>  
<div id="m1" class="magnify"><p>zoom: normal;<p></div>  
<div id="m2" class="magnify"><p>zoom: 200%;<p></div>  
<div id="m3" class="magnify"><p>zoom: 2.9;<p></div>
```



CSS order

- This CSS property specifies the order of the flex item in the grid container or flex
- It is basically used for ordering the flex items
- It is to note that, if the element isn't flexible, then this property will not work
- Syntax
 - order: integer | initial | inherit;

CSS order Property



```
...  
.container {  
    display: flex;  
    background-color: blue;  
    height: 150px;  
    width: auto;  
    flex-wrap: wrap; }  
.....
```



```
</style>  
</head>  
<body>  
<h1> CSS order Property </h1>  
  <div class = "container">  
    <div style = "order: 3"> 1 </div>  
    <div style = "order: 0"> 2 </div>  
    <div style = "order: 0"> 3 </div>  
    <div style = "order: 1"> 4 </div>  
    <div style = "order: -1"> 5 </div>  
    <div style = "order: 2"> 6 </div>  
    <div style = "order: 1"> 7 </div>  
    <div style = "order: -2"> 8 </div>  
  </div>  
</body>  
</html>
```

CSS Descendant Selector



- The CSS descendant selector is used to match the descendant elements of a particular element
- Syntax
 - selector1 selector2 { /* property declarations */ }

```
<style>
```

```
div p { background-color: lightblue;  
font-weight: bold; }
```

```
</style>
```

```
..
```

```
<div>
```

```
<p> This is 1st paragraph in the div. </p>
```

```
<p> This is 2nd paragraph in the div. </p>
```

```
<div>
```

```
This is second div in the first div
```

```
<p> This is the paragraph in second div. It will also be affected. </p>
```

```
</div>
```

```
</div>
```

```
<p> Paragraph 4. It will not be affected because it is not in the div. </p>
```

This is 1st paragraph in the div.

This is 2nd paragraph in the div.

This is second div in the first div

This is the paragraph in second div. It will also be affected.

Paragraph 4. It will not be affected because it is not in the div.

CSS Clip

- This CSS property specifies the visible area of an element
 - It applies to absolutely positioned elements (position: absolute;)
- Syntax
 - clip: auto | shape | initial | inherit;



```
....  
<style type = "text/css">  
  div {  
    background: url(jtp.png);  
    clip: rect(0px, 150px, 250px, 0px);  
    border:3px solid red;  
    height:200px;  
    width: 250px;  
    position: absolute;  
  }  
</style>  
....  
<body>  
  <div></div>  
...
```



clip: auto; property



CSS clip-path



- This CSS property is used to create a clipping region and specifies the element's area that should be visible
 - The area inside the region will be visible, while the outside area is hidden
- Anything outside the clipping will be clipped by browsers, including borders, text-shadows, and many more.
- Syntax
 - `clip-path: <clip-source> | [ <basic-shape> || <geometry-box> || none`
- **clip-source:** It is a url that reference to an SVG `<clippath>` element
- **basic-shape:** It clips the element to a basic shape. It has four basic shapes: **circle**, **ellipse**, **polygon** and **inset**.
- **geometry-box:** The `<geometry-box>` defines the reference box for the basic shape
 - If it is defined with the combination of the `<basic-shape>`, then it will act as the reference box for the clipping done by the `<basic-shape>`
- **geometry-box** can have the below values:
 - `margin-box`, `border-box`, `padding-box`, `content-box`, `fill-box`, `stroke-box`, `view-box`

CSS clip-path example

Clip-path property example

circle



`clip-path: circle(50%);`



`clip-path: circle(60% at 70% 20%);`

Clip-path property example

polygon



Diamond shape polygon

`clip-path: polygon(50% 0%, 100% 50%, 50% 100%, 0% 50%);`



Star shape polygon

`clip-path: polygon(50% 0%, 61% 35%, 98% 35%, 68% 57%, 79% 91%, 50% 70%, 21% 91%, 32% 57%, 2% 35%, 39% 35%);`

CSS clip-path example

```
<style>
img.example { animation: anime 4s infinite;
border: 5px dashed red; }

@keyframes anime {
0% { clip-path: polygon(0 0, 100% 0, 100% 80%, 0% 70%); }
20% { clip-path: polygon(40% 0, 50% 0, 100% 100%, 0% 80%); }
40% { clip-path: polygon(0 0, 60% 72%, 100% 100%, 0 35%); }
60% { clip-path: polygon(70% 0, 20% 0, 100% 100%, 0% 80%); }
80% { clip-path: polygon(0 70%, 60% 0, 100% 32%, 0 40%); }
100% { clip-path: polygon(0 0, 60% 0, 100% 100%, 0% 30%); }
}
</style>
<body>
..
<h2> Animation in Clip-path property </h2>
...
```

Clip-path property example



clip-path: ellipse(50% 35%);



clip-path: ellipse(50% 65% at 70% 50%);

Clip-path property example



clip-path: inset(20% 25% 20% 10%);



clip-path: inset(45% 0% 33% 10% round 10px);

CSS Superscript and Subscript



- In CSS, the vertical-align property is used to specify the superscript and subscript of text using.
 - CSS property specifies the vertical alignment of the text
- Syntax
 - vertical-align: baseline | super | sub ;

```
.....
#super{ vertical-align:super;
        font-size: medium;}
p{ font-weight: bold;
   font-size: 20px; }
</style>
<h1> Using vertical-align: super; </h1>
<p> Exponen ts (powers of a number), mathematical
equations or formulae are the common uses of
superscripted text. </p>
<h3>x<span id="super">2</span>+ y<span
id="super">2</span>= r<span id="super">2</span></h3>
<h3> (a + b)<span id="super">2</span>= a<span
id="super">2</span> + b<span id="super">2</span> + 2ab
</h3>
<h3>5<span id="super">th</span></h3>...
```

Using vertical-align: super;

Exponen ts (powers of a number), mathematical equations or formulae are the common uses of superscripted text.

$$x^2 + y^2 = r^2$$

$$(a + b)^2 = a^2 + b^2 + 2ab$$

5th

CSS Variables

- The CSS variables are used to add the values of custom property to our web page that can be reused throughout the document
- These entities are set using the custom property notation and can be accessed by using the `var()` function
- The variables store the values and have a scope in which they can be used
- Syntax to define variable: `element { --main-color: brown; }`
- The names of the custom properties are case-sensitive
- The **`var()` function** in CSS is used to insert the custom property value. The name of the variable can be passed as the argument to the `var()` function
 - Syntax `var( --custom-name, value )`
- The `var()` function only allows two arguments that are defined as follows:
 - **`--custom-name`**: This parameter accepts the name of the custom property. It must begin with the two dashes (`--`). It is the required parameter
 - **`value`**: It is an optional parameter and accepts the fallback value. It is used as the substitution when the custom property is invalid



CSS Variables example



```
.....
<style>
  :root { --bg-color: lightgreen; }
  body { background-color: var(--bg-color);
        text-align: center; }
  h1 { color: var(--text-color, blue); /* --text-color is not set,
so the fallback value 'blue' will be used */ }
  div { color: var(--text-color, blue);
        font-size: 30px; }
</style>
</head>

<body>
  <h1>Welcome to the javaTpoint.com</h1>
  <div>
    This is an example of CSS variables
    <h3>--bg-color: lightgreen;</h3>...
```

Welcome to the javaTpoint.com

This is an example of CSS variables

--bg-color: lightgreen;

CSS Variables example

- Use of calc() with the var()



```
<style>
:root {
--bg-color: lightgreen;
--extra-padding: 1.2em;
--txt-size: 90px;
}
body { background-color: var(--bg-color); }
h1 {
color: var(--text-color, blue); /* --text-color is not set, so the
fallback value 'blue' will be used */
font-size: calc(var(--txt-size) - 20px);
}
div {
color: var(--text-color, blue);
font-size: 30px;
border: 8px ridge red;
padding: calc(var(--extra-padding) + 20px);
}
.....
<body>
<h1>Welcome to the World</h1>
<div> This is an example of using the calc() function with the
var() function </div>....
```

**Welcome to the
World**

This is an example of using the calc()
function with the var() function

CSS page break



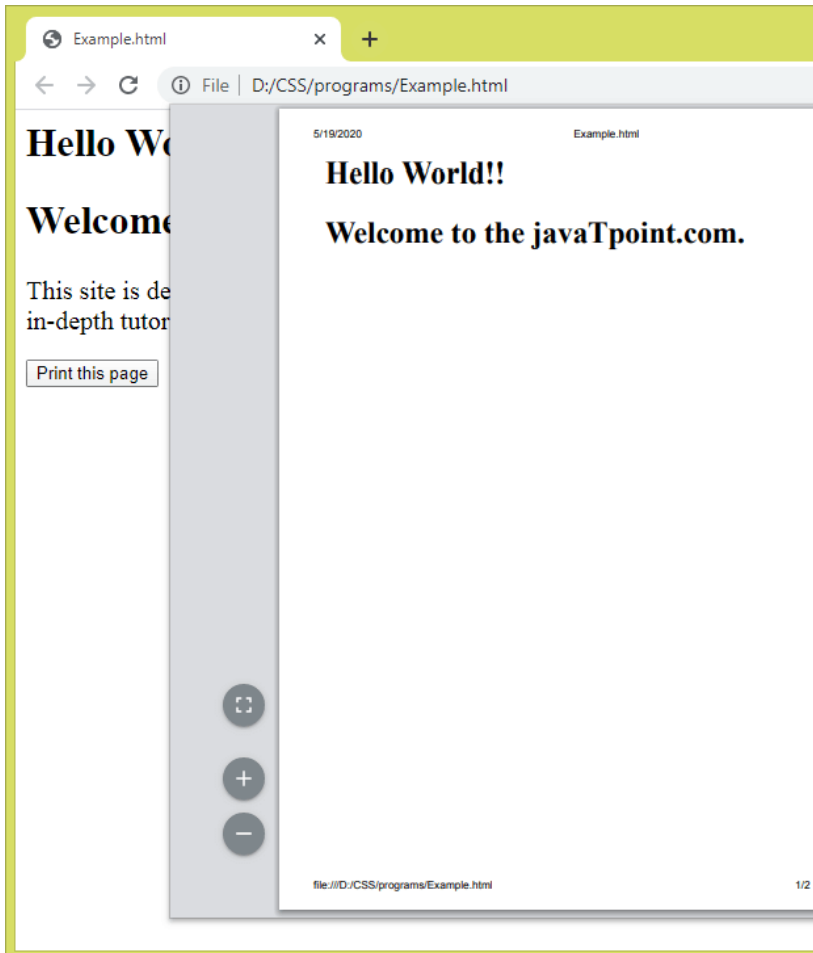
- **page-break-before property:** used to add the page break before the element, when printing the document
 - We cannot use this CSS property on absolutely positioned elements or an empty <div> element that does not generate a box.
- Syntax
 - page-break-before: auto | always | left | right | avoid | initial | inherit;

Values	Description
auto	It is the default value that inserts a page break before the element, if necessary
always	This value always forces a page break before the specified element
avoid	It is used to avoid a page break before the element whenever possible
left	This value forces either one or two page breaks before the element so that the next page will be depicted as the left-hand page
right	The right value forces either one or two page breaks before the element so that the next page will be depicted as the right-hand page
initial	It sets the property to its default value
inherit	If this value is specified, the corresponding element uses the computed value of its parent element page-break-before property

CSS page-break-before example



- Using the right value



```
....
<style type = "text/css">
  div{    font-size: 20px;
          page-break-before: right; }
</style>
.....
<body>
  <div>
    <h2>Hello World!!</h2>
    <h2>Welcome to the javaTpoint.com.</h2>
  </div>
  <div>
    This site is developed so that students may learn computer
    science related technologies easily. The javaTpoint.com is
    committed to providing easy and in-depth tutorials on
    various technologies. No one is perfect in this world, and
    nothing is eternally best. But we can try to be better.
  </div>  <br>
  <button onclick = "func()">Print this page</button>
  <script>
    function func() { window.print(); }
  </script>....
```

CSS page-break-inside property

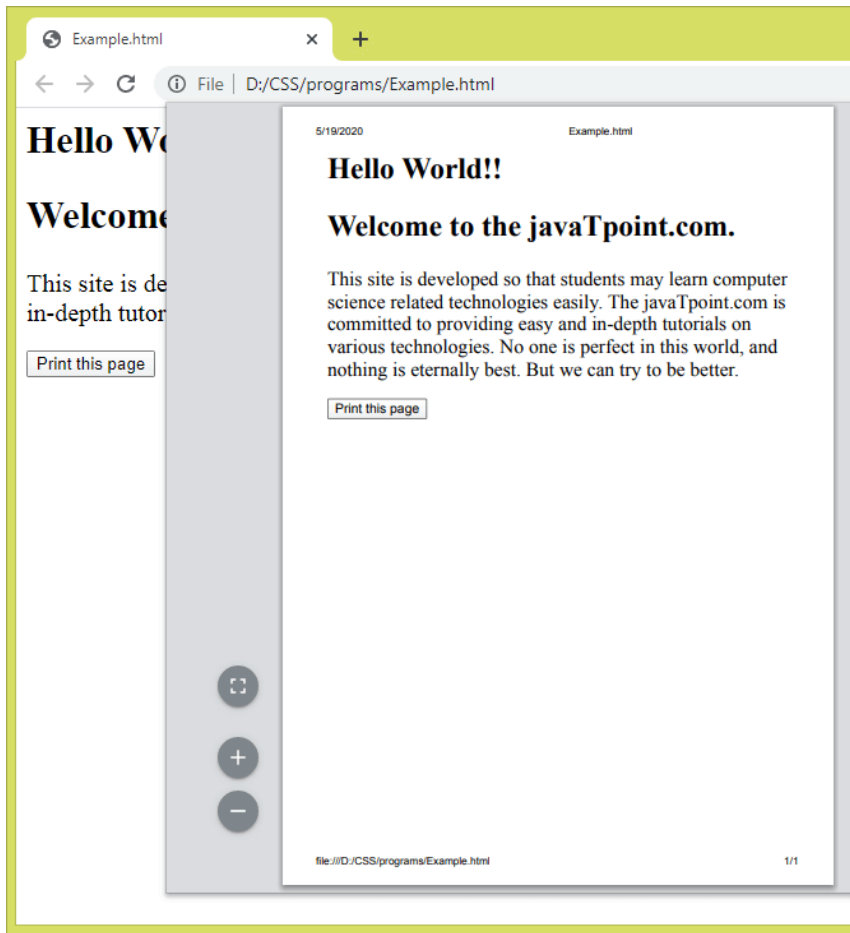


- **page-break-inside** property is used to specify the page break inside the element, when printing the document
- Syntax
 - page-break-inside: auto | avoid | initial | inherit;

Values	Description
auto	It is the default value that inserts a page break inside the element, if necessary.
avoid	It is used to avoid a page break inside the element whenever possible.
initial	It sets the property to its default value.
inherit	If this value is specified, the corresponding element uses the computed value of its parent element page-break-inside property.

CSS page-break-inside example

- Using the avoid value



```
....
<style type = "text/css">
    div{
        font-size: 20px;
        page-break-inside: avoid;
    }
</style>
</head>
<body>
    <div>
        <h2>Hello World!!</h2>
        <h2>Welcome to the javaTpoint.com.</h2>
    </div>
    <div>
        This site is developed so that students may learn computer
        science related technologies easily. The javaTpoint.com is
        committed to .....in this world, and nothing is eternally best.
        But we can try to be better.
    </div>
    <br>
    <button onclick = "func()">Print this page</button>

    <script> function func() { window.print(); } </script>
```

CSS page-break-after property



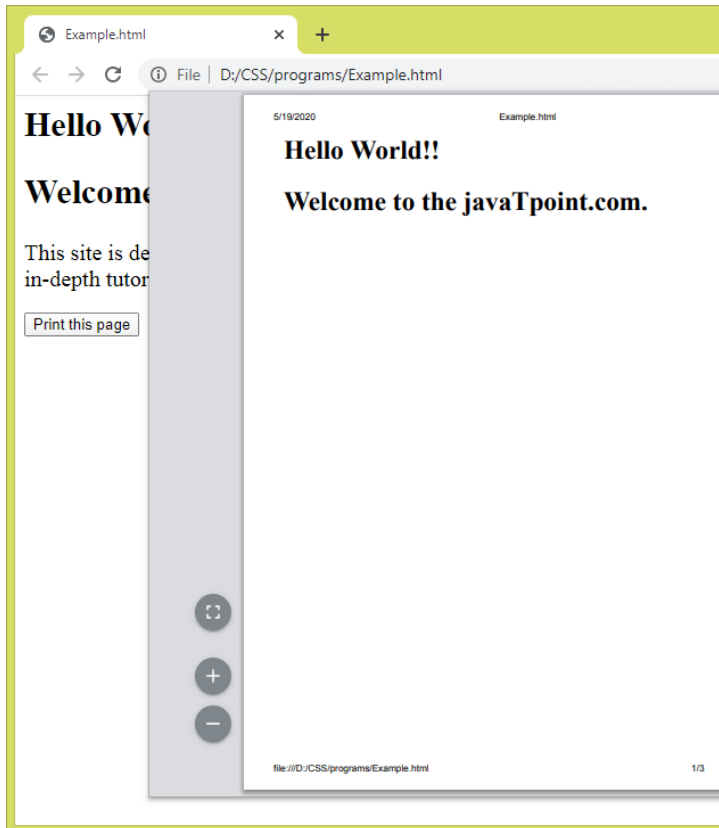
- **page-break-after** property is used to adjust the page break after the element when printing the document
- **Syntax**
 - page-break-after: auto | always | left | right | avoid | initial | inherit;

Values	Description
auto	It is the default value that inserts a page break after the element, if necessary.
Always	It always forces a page break after the specified element.
avoid	It is used to avoid a page break after the element whenever possible.
left	It forces either one or two page breaks after the specified element so that the next page will be depicted as the left-hand page.
right	It forces either one or two page breaks after the specified element so that the next page will be depicted as the right-hand page.
Initial	It sets the property to its default value.
Inherit	If this value is specified, the corresponding element uses the computed value of its parent element.

CSS page-break-after example



- Using the always value



```
....
<style type = "text/css">
    div{ font-size: 20px;
        page-break-after: always; }
</style>
.....
<div>
    <h2>Hello World!!</h2>
    <h2>Welcome to the javaTpoint.com.</h2>
</div>
<div>
    This site is developed so that students may learn
    computer science .....this world, and nothing is
    eternally best. But we can try to be better.
</div>
<br>
<button onclick = "func()">Print this page</button>
<script> function func() { window.print(); }
</script>
```

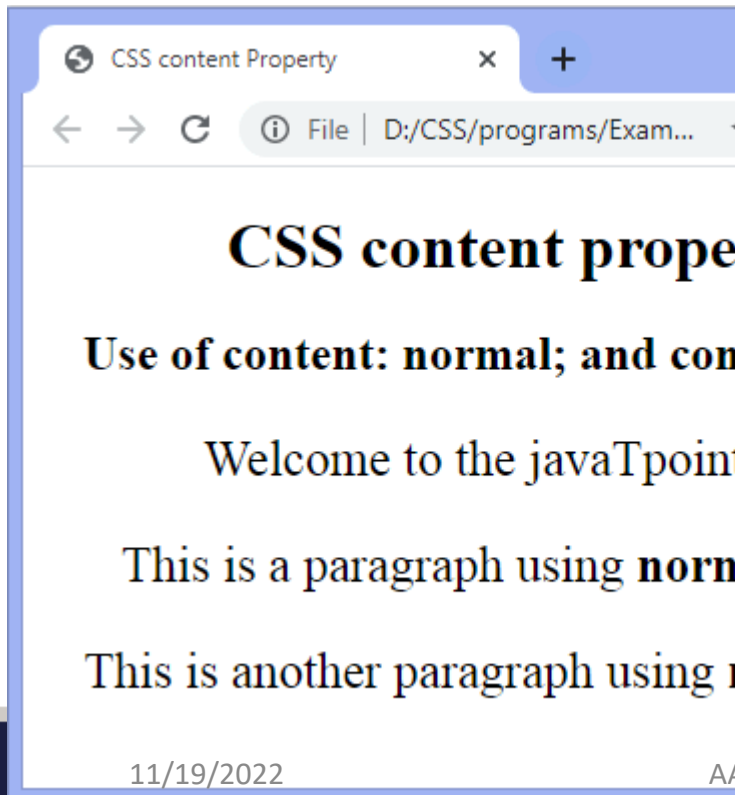
CSS content property

- This CSS property generates dynamic content. It can be used with the pseudo-elements **::before** and **::after**
- Syntax
 - content: normal | none | counter | string | attr | open-quote | close-quote | no-close-quote | no-open-quote | url | initial | inherit;



Values	Description
normal	It is used to set the default value
none	This value does not set the content.
counter	It sets the content as the counter. It is generally a number. It is displayed by using the counter() or counters() function.
string	It is used to set any string value. It should always be enclosed within quotation marks. It generates any string after or before the HTML element.
attr	It inserts the value of the specified attribute after or before the element. If the selector does not have a particular attribute, then an empty string will be inserted.
open-quote	It is used to insert the opening quotation mark, or it sets the content to be an opening quote.
close-quote	It is used to insert the closing quotation mark, or it sets the content to be a closing quote.
no-close-quote	If the closing quotes are specified, then it is used to remove the closing quote from the content.
no-open-quote	If the opening quotes are specified, then it is used to remove the opening quote from the content.
url	It is used to set the content to some media, which could be an image, video, audio, and many more.
Initial	It is used to set the property to its default value.
Inherit	It inherits the property from its parent element.

CSS content property example



```
.....
<style>
body{ text-align: center; }
p{ font-size: 25px; }
p::before { content: "Welcome "; }
#para::before { content: normal; }
#para1::before { content: none; }
</style>
.... <h1> CSS content property </h1>
<h2> Use of content: normal; and content: none; </h2>
<p> to the javaTpoint.com </p>
<p id = "para"> This is a paragraph using <b>normal</b> value. </p>

<p id = "para1"> This is another paragraph using <b>none</b> value.
</p> .....
```



CSS content example using string and url value

CSS content property

Use of content: string; and content: url();

Hello World. Welcome to the javaTpoint.com



This is a paragraph using the **url()** value.



This is another paragraph using the **url()** value.

```
...
p{ font-size: 25px;    }
p::before { content: "Hello World. Welcome "; }
#para::before { content: url("img.png"); }
#para1::before { content: url("img.png"); }
</style>

....
<h1> CSS content property </h1>
<h2> Use of content: string; and content: url();
</h2>
<p> to the javaTpoint.com </p>
<p id = "para"> This is a paragraph using the
<b>url()</b> value. </p>
<p id = "para1"> This is another paragraph using
the <b>url()</b> value. </p>    ..
```

CSS content

example using open-quote and close-quote value



CSS content property

Use of content: open-quote; and content: close-quote;

“ Welcome to the javaTpoint.com ”

“ This is another paragraph. ”

Using no-open-quote; and no-close-quote

CSS content property

Use of content: no-open-quote; and content: no-close-quote;

" Welcome to the javaTpoint.com "

This is another paragraph

....

```
body{ text-align: center; }
```

```
p{ font-size: 25px;}
```

```
p::before {content: open-quote;}
```

```
p::after { content: close-quote }
```

```
</style>
```

```
</head>
```

..

```
<h1> CSS content property </h1>
```

```
<h2> Use of content: open-quote; and  
content: close-quote; </h2>
```

```
<p> Welcome to the javaTpoint.com </p>
```

```
<p> This is another paragraph. </p>
```

```
</body>
```

```
</html>
```

CSS content example using attr()



```
....  
<style>  
  body{ text-align: center; }  
  a::after { content: attr(href); }  
</style>
```

```
.....  
<h1> CSS Content property </h1>  
<h2> The following link is displayed by using the  
<b>attr()</b> </h2>  
  <a href= https://www.javatpoint.com>Click  
here! </a> ...
```

CSS Content property

The following link is displayed by using the attr()

[Click here! https://www.javatpoint.com](https://www.javatpoint.com)

CSS object-fit property



- This CSS property specifies how a video or an image is resized to fit its content box
- It defines how an element fits into the container with an established width and height
- Syntax
 - object-fit: fill | contain | cover | none | scale-down | initial | inherit;
- **fill**: It is the default value. Using this value, the entire object fills in the container. The replaced content is sized to fill the content box. If the aspect ratio of the object doesn't match the aspect ratio of the box, the object will be squished or stretched to fit in the box
- **contain**: It resizes the content of an element to stay contained inside the container. It fits the image in the width and height of the box while maintaining the aspect ratio of the image. The replaced content is scaled for maintaining the aspect ratio while fitting within the content box of the element.
- **cover**: It resizes the content of an element to cover its container. It fills the entire box with the image. If the aspect ratio of the object is not matched with the aspect ratio of the box, it clips the object to fit
- **none**: It does not resize the replaced content. The image retains its original size and ignores the height and width of the parent
- **scale-down**: It sized the content as none or as contain. It results in the smallest object size. It finds the smallest concrete object size by comparing the none and contain

CSS object-fit property example



Original image



object-fit: fill;



object-fit: contain;



object-fit: cover;



object-fit: scale-down;



```
..
#img1{ width: 300px;
height: 300px;
border: 7px solid red;
#obj { width: 500px;
height: 500px;
object-fit: fill;
border: 7px solid red; }
#left{ float: left;
text-align: center;
padding-left: 200px; }
#center{ float: center;
text-align: center; }
.....
<h1> Example of Object-fit property </h1>
<div id = "left">
<h2> Original image </h2>
<img id = "img1" src = "forest.jpg">
</div>
<div id= "center">
<h2> object-fit: fill; </h2>
column-count</b> and <b>column-width</b> to the default browser values.                     |
| column-width | It is used to set the minimum width for columns. However, the actual width of the column may be narrower or wider based on the available space. |
| column-count | It specifies the maximum number of columns.                                                                                                     |
| Initial      | It is used to set the property to its default value.                                                                                            |
| Inherit      | It inherits the property from its parent element.                                                                                               |

# CSS columns example



```
.....
.div1 { columns: 100px 4;
 border: solid 2px black;
 font-size: 20px; }
.div2 { columns: 100px 6;
 border: solid 2px black;
 font-size: 20px; }
```

```
</style>
```

```
....
```

```
<h1> The columns Property </h1>
```

```
<h4> The columns ...-count </h4>
```

```
<h3> Set the column-width to 100px, and
count 4 </h3>
```

```
<div class="div1">
```

Hi, Welcome.....easy and  
depth ....ots of tutorials that are easy to understand a  
No one is perfect in this world, and nothing is eternal  
we can try to be better.

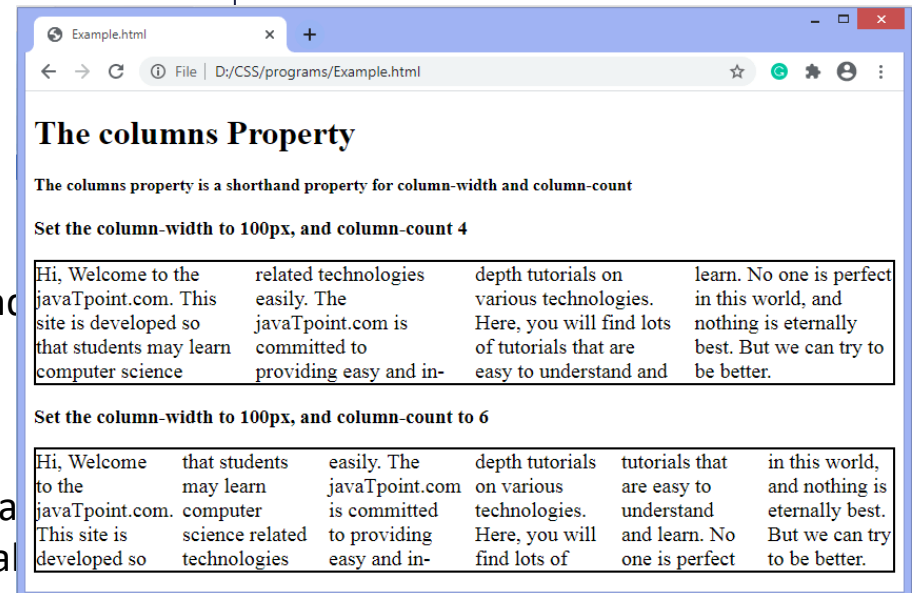
```
</div>
```

```
<h3>Set the column-width to 100px, and column-
count to 6 </h3>
```

```
<div class="div2">
```

Hi, Welcome to the... This site is ..to providing easy and in-  
depth tutorials on ...best. But we can try to be better.

```
</div>
```



# CSS pointer-events property



- This CSS property specifies whether or not an element shows some action when the pointer event is triggered upon it
- The pointer-events are triggered by touch, stylus, mouse click, and many more
- Syntax
  - `pointer-events: none | auto | initial | inherit;`

```
p{
 font-size: 20px;
 pointer-events: none;
}
```

```
p{
 font-size: 20px;
 pointer-events: auto;
}
```

# CSS hyphens property



- This CSS property defines how the word is hyphenated if it is too long or when the text wraps across multiple lines
- Syntax
  - hyphens: none | manual | auto | initial | inherit;
- **none**: This value does not hyphenate the words. It never hyphenates the words at line breaks or even if the word is too long.
- **manual**: It is the default value that hyphenates the word only when the characters in the word suggest hyphenation opportunities
  - The two Unicode characters are defined below, which can be used manually to specify the possible line breakpoints in the text
    - **U+2010 (HYPHEN)** - It is the 'hard' hyphen character that specifies the visible line-break opportunity. The hyphen is rendered, even if the line is not broken at that point.
    - **U+00AD (SHY)** - It is an invisible 'soft' hyphen. It is not visibly rendered; instead, it spots the place where the word should be required to break. In html, for a soft hyphen, we can use &shy;
- **auto**: In this value, the algorithm decides where the words are hyphenated

# CSS hyphens example

```
....
<style>
div { width: 50px;
border: 3px solid blue;
background-color: lightblue; }
.none{ hyphens: none; }
.manual{ hyphens: manual; }
.auto{ hyphens: auto; }
</style>
.....
<h2> Example of the hyphens property </h2>

<h3> hyphens: none; </h3>
<div class="none"> It is veryvery looooo-
ooooooong word. </div>
<h3>hyphens: manual</h3>
<div class="manual"> It is veryvery looooo-
oooooooong word. </div>
<h3>hyphens: auto</h3>
<div class="auto">
It is very-very looooo-ooooo-oong word.
</div>
```

## Example of the hyphens property

**hyphens: none;**

It is  
veryvery  
loooooo-oong  
word.

**hyphens: manual**

It is  
very-  
very  
looooo-  
oooo-  
oong  
word.

**hyphens: auto**

It is  
very-  
very  
looooo-  
oooo-  
oong  
word.

# CSS Position Offset Properties



- The Offset CSS Properties of a box are **top, left, right, bottom**
- They are applied to the containing box in the following positioning model in order to position a box
  - relative, absolute or fixed
- They are used then only when the box has one of the following position value:
  - relative, absolute or fixed
- Example

```
.up {
 position:relative;
 bottom:10px;
}
```

.....

An example on how to move a words

```
Up 10
pixels
```

An example on how to move a words Up 10 pixels

# CSS Max-width, Min-width, Max-height, and Min-height



- The **max-width** property lets you specify an element's box maximum width
  - This means that element's box can increase in width until it reaches a specific absolute or relative unit, at which point it should fix its width to that unit
- The **min-width** property specifies the minimum width

## Example using max-width

```
//The card can not get larger
than 350px.
.card{ margin:0 auto;
 padding:1.5em;
 width:80%;
 max-width:350px;
 height:50%;
 background: #FFFFFF;
 box-shadow: 0px 0px 5px
 rgba(0, 0, 0, 0.3);
 border-radius:4px;
 overflow:hidden;
}
```

## Example using min-width

```
//Here the card element can
not get smaller than 250px
.card{ margin:0 auto;
 padding:1.5em;
 width:80%;
 max-width:350px;
 //here we set the min-width
property
 min-width : 250px;
 height:50%;
 background: #FFFFFF;
 box-shadow: 0px 0px 5px
 rgba(0, 0, 0, 0.3);
 border-radius:4px;
 overflow:hidden;
}
```

# CSS Max-width, Min-width, Max-height, and Min-height



- **max-height:** this property operates similarly to the max-width property, but it affects the height instead of the width
- **min-height** property provides a minimum height for an element

## Example using max-height

```
// it fixes the height of an element to a
specified unit, effectively stopping it from
increasing in height
.card{
 margin:0 auto;
 padding:1.5em;
 width:80%;
 max-width:350px;
 height:70%;
 //here we set the max-height for the card.
 max-height: 400px;
 background: #FFFFFF;
 box-shadow: 0px 0px 5px rgba(0, 0, 0, 0.3);
 border-radius:4px;
 overflow:hidden;
}
```

## Example using min-height

```
.element{
 height:40vh;
 min-height:200px;
}
```

# CSS cubic-bezier()



- It is an inbuilt function in CSS that defines a Cubic Bezier curve
- This CSS function is the transition timing function, which is used for the smooth and custom transitions
- It is defined by the four points (that are P0, P1, P2, and P3)
  - The points P0 and P3 (called as anchors) are the starting and the ending points
  - The points P1 and P2 (called as handles) are the middle points
- the points P0 and P3 are always in the same spot. P0 is at (0,0) that represents the initial state and initial time, and P3 is at (1,1), which represents the final state and the final time
- Syntax
  - `cubic-bezier( x1, y1, x2, y2 )`

# CSS cubic-bezier() example



```
...
.jtp {
 width: 200px;
 height: 50px;
 background: blue;
 transition: width 3s;
 animation-timing-function: cubic-
 bezier(.59,1.01,0,.01)
}
div:hover { width:300px; }
p{ font-size: 20px;
color: darkviolet; }
....
<h1> The cubic-bezier() Function </h1>
<p> Move your mouse over the below div element,
to see the transition effect. </p>
<div class = "jtp">.....
```

# CSS quotes



- The quotes property in CSS specifies the type of quotation mark for the quotation used in the sentence
- Syntax
  - quotes: none | string | initial;
- **none**: It is the default value that specifies that the open-quote and close-quote values of the content property do not generate any quotation marks.
- **string**: This value specifies the type of quotation mark to be used in sentence. The first pair represents the outer-level quotation; the second pair indicates the first nested level, the third pair indicates the third level, and so on

	Description	Entity number
"	double quote	\0022
'	single quote	\0027
„	double quote (double low-9)	\201E
«	double, left angle quote	\00AB
»	double, right angle quote	\00BB

	Description	Entity number
‹	single, left angle quote	\2039
›	single, right angle quote	\203A
'	left quote (single high-6)	\2018
'	right quote (single high-9)	\2019
“	left quote (double high-6)	\201C
”	right quote (double high-9)	\201D

# CSS quotes example using string

```
...
body{ text-align: center; }
h1{ color: blue; }
p{ font-size: 20px; }
#j1 { quotes: '«' '»'; }
#j2 { quotes: '“' '”'; }
#j3 { quotes: '„' '„'; }
#j4 { quotes: '‹' '›'; }
#j5 { quotes: '„' '„'; }
#j6 { quotes: '‹' '›' '‹' '›'; }
#j7 { quotes: '\2018' '\2019'; }
#j8 { quotes: '\2039' '\203A'; }
#j9 { quotes: '\201C' '\201E'; }
#j10 { quotes: '\201D' '\201E'; }
#j11 { quotes: '\0022' '\201E'; }
#j12 { quotes: '\201C' '\201D'; }
#j13 { quotes: initial; }
```

```
<h1> Example of the quotes:string; </h1>
<p><q id="j1"> javaTpoint.com </q></p>
<p><q id="j2"> javaTpoint.com </q></p>
<p><q id="j3"> javaTpoint.com </q></p>
<p><q id="j4"> javaTpoint.com </q></p>
<p><q id="j5"> javaTpoint.com </q></p>
<p><q id="j6"> javaTpoint.com </q></p>
<p><q id="j7"> javaTpoint.com </q></p>
<p><q id="j8"> javaTpoint.com </q></p>
<p><q id="j9"> javaTpoint.com </q></p>
<p><q id="j10"> javaTpoint.com </q></p>
<p><q id="j11"> javaTpoint.com </q></p>
<p><q id="j12"> javaTpoint.com </q></p>
<p><q id="j13"> javaTpoint.com </q></p> ...
```

## Example of the quotes:string;

```
< javaTpoint.com >
‘ javaTpoint.com ’
” javaTpoint.com „
« javaTpoint.com »
“ javaTpoint.com ”
< javaTpoint.com >
‘ javaTpoint.com ’
< javaTpoint.com >
“ javaTpoint.com „
” javaTpoint.com „
" javaTpoint.com „
“ javaTpoint.com ”
“ javaTpoint.com ”
```

# CSS quotes example using :lang pseudo-class



```
.....
p{
font-size: 25px;
color: red;
}
:lang(en) { quotes: '"" '""'; }
:lang(fr) { quotes: '\201D' '\201E' ; }
</style>
```

```
.....
<p><q lang = "en"> Welcome to the javaTpoint.com. </q>

<q lang = "fr"> Ce site est développ   pour que les   tudiants
puissent apprendre facilement les technologies li  es   
l'informatique.</q>

The javaTpoint.com is committed to provide easy and in-
depth tutorials on various technologies. </q></p>
</body>
</html>
```

“ Welcome to the javaTpoint.com. ”  
” Ce site est d  velopp   pour que les   tudiants  
puissent apprendre facilement les technologies li  es     
l'informatique. „  
The javaTpoint.com is committed to provide easy  
and in-depth tutorials on various technologies.

# CSS transform-origin property



- This CSS property is used to change the position of transformed elements
- The transform-origin property must be used with the transform property
- The 2d transformation can change the x-axis and y-axis of the element, whereas the 3D transformation can change the z-axis along with the x-axis and y-axis.
- This property can be specified by using one, two, or three values
  - The first value affects the horizontal position
  - The second value affects the vertical position, and
  - The third value indicates the position of the z-axis. The third value can also work on 3D transformations and cannot be defined using a percentage
- If we specify only one value, the value must be a length or percentage, or one of the keyword values left, center, right, top, and bottom.
- If we specify two values, the first value must be a length or percentage or the keyword values left, right, and center. The second value must be a length or percentage or one of the keyword values center, top, and bottom
- When we specify three values, then the first two values are same as the two-value syntax, but the third value represents the z-offset, so it must be a length
- The default value of the transform-origin property is 50% 50%, which represents the center of the element

# CSS transform-origin cont'd



- Syntax
  - transform-origin: x-axis y-axis z-axis | initial | inherit;

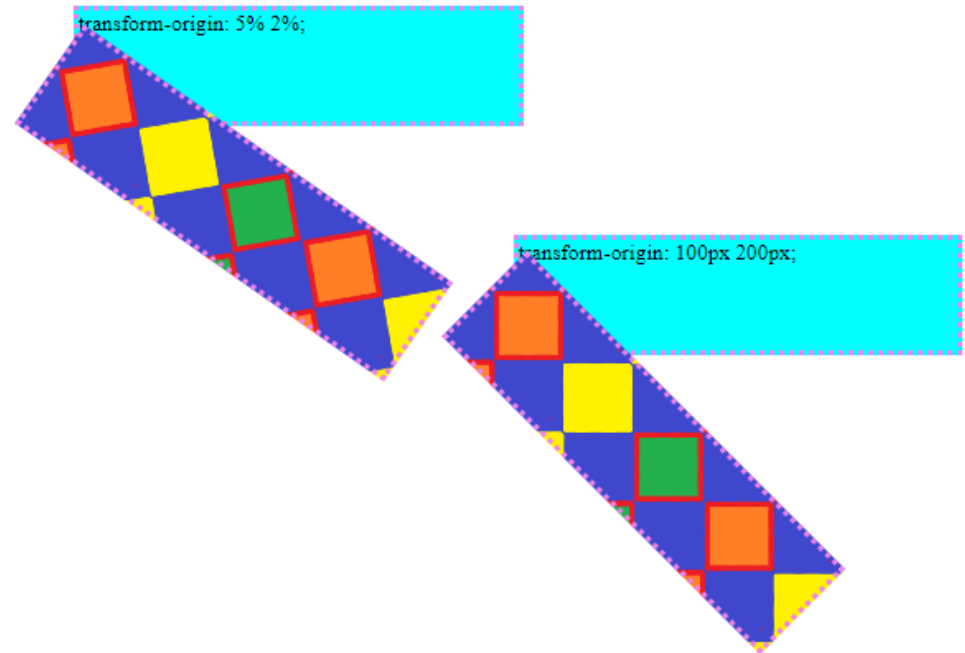
Values	Description
<b>x-axis</b>	It represents the horizontal position. This value specifies the position of the view at x-axis. Its possible values are length, percentage, <b>left</b> , <b>right</b> , and <b>center</b> .
<b>y-axis</b>	It represents the vertical position. This value specifies the position of the view at y-axis. Its possible values are <b>length</b> , <b>percentage</b> , <b>top</b> , <b>bottom</b> , and <b>center</b> .
<b>z-axis</b>	This value is used with the 3D transformations. This value specifies the position of the view at z-axis. It can be defined using the <b>length</b> values. It does not allow the percentage values.
<b>initial</b>	It sets the property to its default value.
<b>inherit</b>	It inherits the property from its parent element.

# CSS transform-origin example



```
...
<style>
div{
height: 100px;
width: 400px;
border: 5px dotted violet;
font-size: 20px;
}
.outer { margin: 100px;
background-color: cyan; }
.box {
background: url("diamond.png");
transform: rotate(35deg);
transform-origin: 5% 2%;
}
.outer1{ margin-left: 500px;
background-color: cyan; }
.box1 { background: url("diamond.png");
transform: rotate(45deg);
transform-origin: 5% 2%; }
.....
<body>
<h1> Example ..Property </h1>
<div class="outer"> transform-origin: 5% 2%;
<div class="box"></div>
</div>
<div class="outer1"> transform-origin: 100px 200px;
<div class="box1"></div>
</div>.....
```

## Example of the CSS transform-origin Property

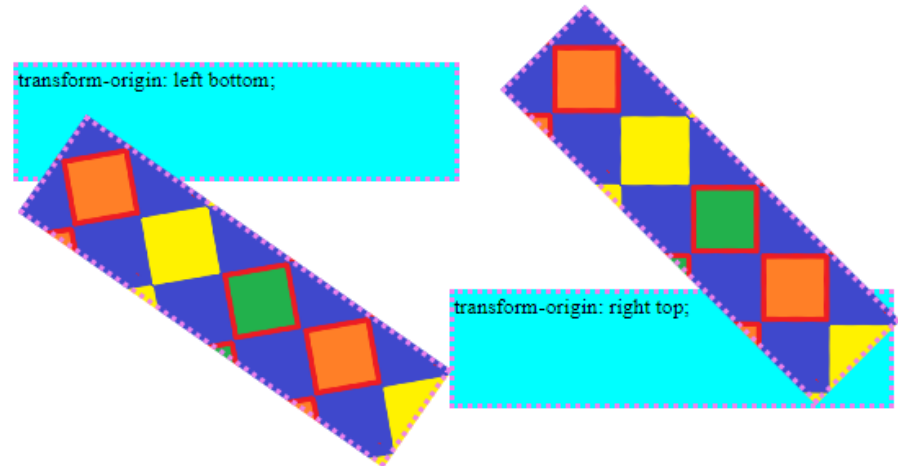


# CSS transform-origin example

```
...
div{ height: 100px;
width: 400px;
border: 5px dotted violet;
font-size: 20px; }
.outer { margin: 100px;
background-color: cyan; }
.box { background: url("diamond.png");
transform: rotate(35deg);
transform-origin: left bottom; }
.outer1{ margin-left: 500px;
background-color: cyan; }
.box1 { background: url("diamond.png");
transform: rotate(45deg);
transform-origin: right top; }
</style>
....
<body>
<h1> Example .. Property </h1>
<div class="outer"> transform-origin: left bottom;
<div class="box"></div>
</div>
<div class="outer1"> transform-origin: right top;
<div class="box1">....
```

**transform-origin** example using keyword values

## Example of the CSS transform-origin Property



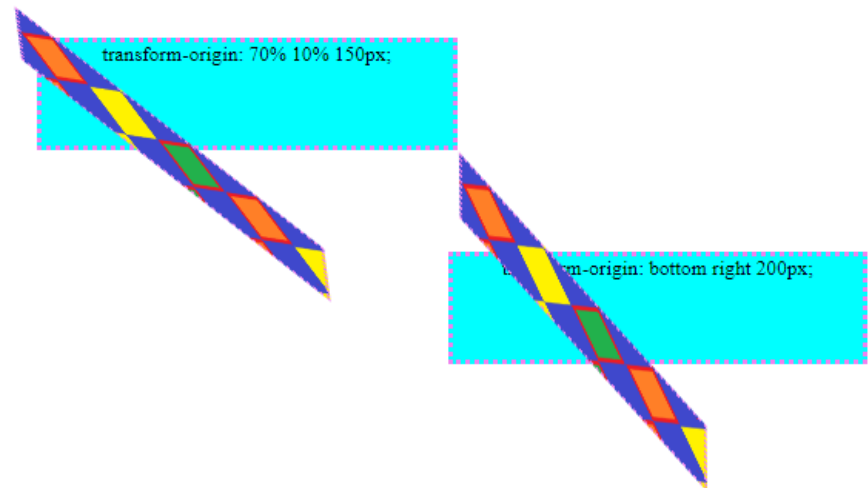
# CSS transform-origin example



```
....
div{ height: 100px;
width: 400px;
border: 5px dotted violet;
font-size: 20px; }
.outer { margin: 100px;
background-color: cyan;
text-align: center; }
.box { background: url("diamond.png");
transform: rotate3d(3, 2, 1, 75deg);
transform-origin: 70% 10% 150px; }
.outer1{ margin-left: 500px;
background-color: cyan;
text-align: center; }
.box1 { background: url("diamond.png");
transform: rotate3d(2, 2, 1, 75deg);
transform-origin: bottom right 200px; }
.....
<body>
<h1> ExampleProperty </h1>
<div class="outer"> transform-origin: 70% 10%
150px;
<div class="box"></div>
</div>
<div class="outer1"> transform-origin: bottom right
200px;
<div class="box1"></div>
</div>.....
```

**transform-origin** example property on the 3D transformed elements

**Example of the CSS transform-origin Property**



# CSS resize property



- This CSS property allows the user to control the resizing of an element just by clicking or dragging the bottom right corner of the element
- This CSS property is used to define how an element is resizable by the user
- It doesn't apply on the block or inline elements where overflow is set to visible. So, to control the resizing of an element, we have to set the overflow other than visible like (overflow: hidden or overflow: scroll)
- Syntax
  - `resize: none | horizontal | vertical | both | initial | inherit;`
- **none:** It is the default value of this property, which does not allow resizing the element.
- **horizontal:** This value allows the user to resize the element's width. It resizes the element in a horizontal direction
  - There is a unidirectional horizontal mechanism for controlling the width of an element
- **vertical:** It allows the user to resize the height of an element. It resizes the element in a vertical direction.
  - There is a unidirectional vertical mechanism for controlling the height of an element
- **both:** It allows the user to resize the width and height of an element. It resizes the element in both horizontal and vertical directions

# CSS resize example

.....  
div {    Example using horizontal value

```
border: 2px solid red;
padding: 20px;
font-size: 25px;
width: 300px;
resize: horizontal;
overflow: auto;
background-color: lightgreen;
color: blue;
```

```
}
```

</style>

.....  
<h1>Example of the resize: horizon  
tal; </h1>

<div>

  <p> This is the div element. </p>

  <p> To see the ...lement. </p>

</div>

...

Example of the resize: horizontal;

This is the div element.

To see the resizing effect, click and drag the  
bottom right corner of this div element.

Example of the resize: both;

This is the div element.

To see the resizing effect, click and drag the bottom right  
corner of this div element.

Example of the resize: vertical;

This is the div element.

To see the resizing effect,  
click and drag the bottom  
right corner of this div  
element.

Example of the resize: none;

This is the div element.

There is not any dragging  
mechanism at the bottom  
right corner because it is the  
**none** value

# CSS transition-delay property



- This CSS property specifies the duration to start the transition effect
- The value of this property is defined as either seconds (s) or milliseconds (ms)
- Syntax
  - transition-delay: time | initial | inherit;
- **time**: It specifies the amount of time (in seconds or milliseconds) to wait before the transition starts

# CSS transition-delay example



```
...
<style>
div{
width: 100px;
height: 100px;
background: lightblue;
transition-property: background-color;
transition-duration: 1s;
transition-delay: 0.5s;

/* For Safari browser */
-webkit-transition-property: background-color;
-webkit-transition-duration: 1s;
-webkit-transition-delay: 0.5s;
}
div:hover { background-color: brown; }
</style>
.....
<div></div>
<p> Move the cursor ..n effect. </p>
```

- Example using the **transition-property**, **transition-duration**, and **transition-delay** properties
- There is a delay of **0.5s** to start the transition effect, i.e., the background color of the div element will be changed after the given amount of time.

# CSS transition-delay example

....

<style>

```
div {
padding:15px;
width: 200px;
height: 200px;
background: lightgreen;
transition: background-
color 1s, width 2s, height 2s, transform 2s;
transition-delay: 1.5ms;
}
```

```
p{ font-size: 20px; }
div:hover {
width: 300px;
height: 300px;
-webkit-transform: rotate(360deg);
transform: rotate(360deg);
background-color: orange;
}
```

</style>

.....

<div>

<h2>JavaTpoint.com</h2>

<p> (Move your cursor on me to see the effect)</p></div>

> ..

## Example

- Transition effects on width, height, and transformation
- There is also a **transition-delay** in the **milliseconds (ms)**, i.e., **2.5ms**.

# Exercise



- Checkbox styling
- Radio button styling
- Navigation bar
- CSS overlay



# Cascading Style Sheet (CSS)

**Internet Programming I: Chapter 3 – Part III**



ADDIS ABABA SCIENCE AND TECHNOLOGY UNIVERSITY  
**Department of Software Engineering**

# CSS Grid



- CSS Grid layout divides a page into major regions
- Grid property offers a grid-based layout system having rows and columns
- It makes the designing of web pages easy without positioning and floating
- Similar to the table, it enables a user to align the elements into rows and columns
  - But compare to tables, it is easy to design layout with the CSS grid
  - We can define columns and rows on the grid by using grid-template-rows and grid-template-columns properties
- Grid Container
  - We can define the grid container by setting the display property to grid or inline-grid on an element
  - Grid container contains grid items that are placed inside rows and columns

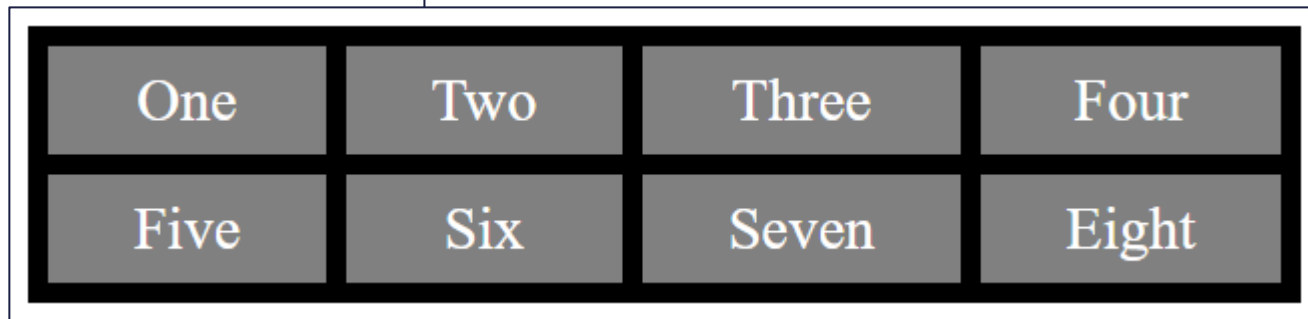
# Grid Container example



```
.main {
 display: grid;
 grid: auto auto / auto auto auto auto;
 grid-gap: 10px;
 background-color: black;
 padding: 10px;
}
.num {
 background-color: grey;
 text-align: center;
 color: white;
 padding: 10px 10px;
 font-size: 30px;
}
```

....

```
<div class="main">
 <div class="num">One</div>
 <div class="num">Two</div>
 <div class="num">Three</div>
 <div class="num">Four</div>
 <div class="num">Five</div>
 <div class="num">Six</div>
 <div class="num">Seven</div>
 <div class="num">Eight</div>
</div>
```



# CSS Grid shorthand properties



- **grid-template-columns:** It is used to specify the size of the columns
- **grid-template-rows:** It is used to specify the row size
- **grid-template-areas:** It is used to specify the grid layout by using the named items
- **grid-auto-rows:** It is used to specify the automatic size of the rows
- **grid-auto-columns:** It is used to specify the automatic size of the columns
- **grid-auto-flow:** It is used to specify how to place auto-placed items and the automatic row size

# CSS Grid Example using shorthand properties



```
.main {
 display: grid;
 grid-template-columns: 1fr 1fr 1fr;
 grid-template-rows: 100px 250px 200px;
 background-color: black;
 grid-gap: 10px;
 padding: 20px;
}
```

```
.num {
 background-color: lightgrey;
 text-align: center;
 padding: 20px 10px;
 font-size: 30px;
}
```

```
</style>
```

```
<div class="main">
 <div class="num">One</div>
 <div class="num">Two</div>
 <div class="num">Three</div>
 <div class="num">Four</div>
 <div class="num">Five</div>
 <div class="num">Six</div>
 <div class="num">Seven</div>
 <div class="num">Eight</div>
</div>
```



# CSS Grid cont'd



- **justify-content property:** used to align the entire grid within the container. It includes values such as:
  - **space-evenly:** It provides equal space in between or around the columns
  - **space-around:** It provides equal space around the columns
  - **space-between:** It gives an equal amount of space between the columns
  - **center:** It is used to align the grid in the middle of the container
  - **start:** It is used to align the grid at the beginning of the container
  - **end:** It is used to align the grid at the end of the container
- **align-content:** property used to align the entire grid within the container vertically
- **Note:** It is noted that the total height of the grid should be less than the height of the container for any effect of the align-content property
- The values of the align-content property are the same as the values of the justify-content property

# justify-content example

```
<style>
.grid-container1 { display: grid;
 justify-content: space-evenly;
 grid-template-columns: 50px 50px 50px;
 grid-gap: 10px;...}
.grid-container2 { display: grid;
 justify-content: space-around;
 grid-template-columns: 50px 50px 50px;
 grid-gap: 10px;...}
.grid-container3 { display: grid;
 justify-content: space-between;
 grid-template-columns: 50px 50px 50px;
 grid-gap: 10px;...}
.grid-container4 { display: grid;
 justify-content: end;
 grid-template-columns: 50px 50px 50px;
 grid-gap: 10px;...}
.grid-container5 {
 display: grid;
 justify-content: start;
 grid-template-columns: 50px 50px 50px;
 grid-gap: 10px;...}
.grid-container6 { display: grid;
 justify-content: center;
 grid-template-columns: 50px 50px 50px;
 grid-gap: 10px;....}
.num {..
```

```
.....</head>
```

```
<body>
```

```
 CONTAINER WITH SPACE-EVENLY VALUE
```

```
<div class="grid-container1">
```

```
 <div class='num'>1</div>
```

```
 <div class='num'>2</div>
```

```
 <div class='num'>3</div>
```

```
</div>
```

```
CONTAINER WITH SPACE-AROUND VALUE
```

```
<div class="grid-container2">
```

```
 <div class='num'>1</div>
```

```
 <div class='num'>2</div>
```

```
 <div class='num'>3</div>
```

```
</div>
```

```
CONTAINER WITH SPACE-BETWEEN VALUE
```

```
<div class="CONTAINER WITH SPACE-EVENLY VALUE"
```

```
 <div class="1 2 3"
```

```
</div>
```

```
CONTAINER WITH SPACE-AROUND VALUE"
```

```
 <div class="1 2 3"
```

```
</div>
```

```
CONTAINER WITH SPACE-BETWEEN VALUE"
```

```
 <div class="1 2 3"
```

```
</div>
```

```
CONTAINER WITH END VALUE"
```

```
 <div class="1 2 3"
```

```
</div>
```

```
CONTAINER WITH START VALUE"
```

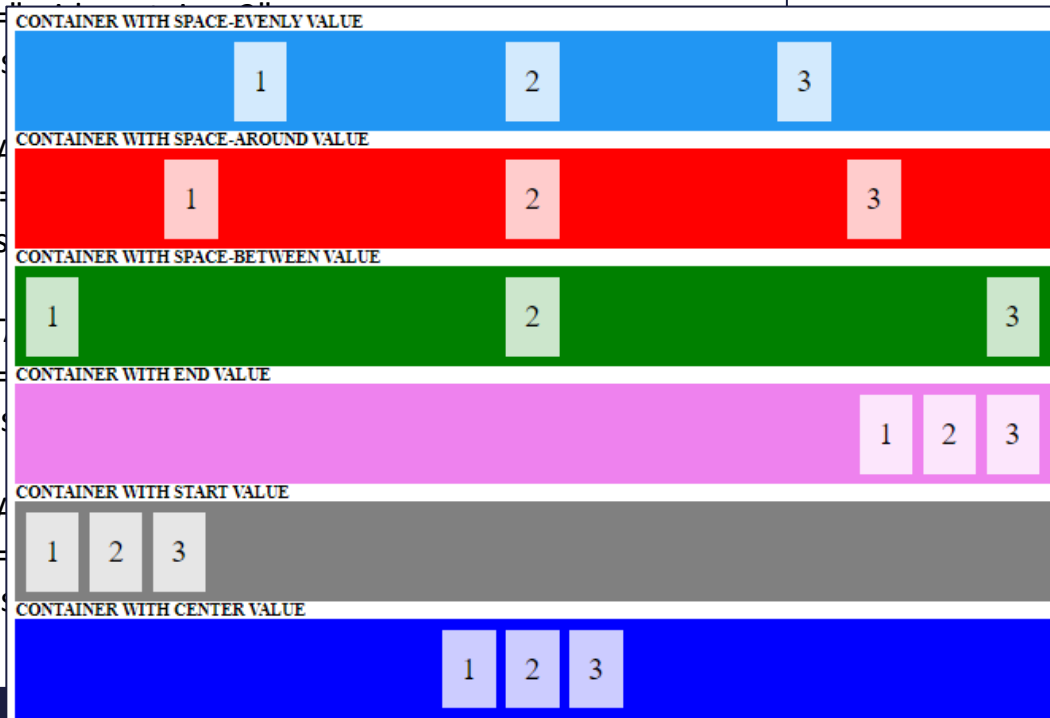
```
 <div class="1 2 3"
```

```
</div>
```

```
CONTAINER WITH CENTER VALUE"
```

```
 <div class="1 2 3"
```

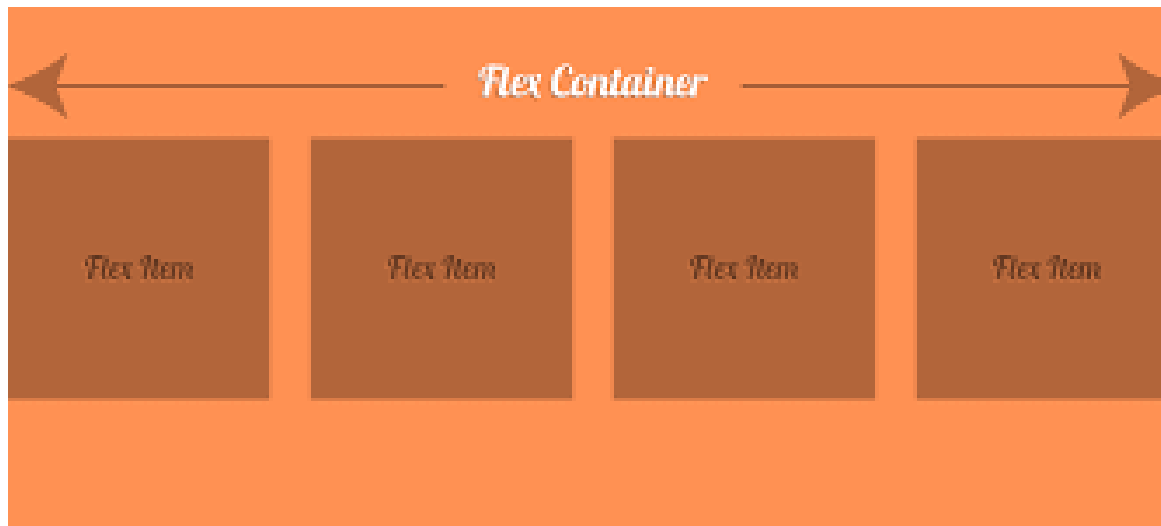
```
</div>...
```



# CSS Flexbox (Flexible boxes)



- The CSS3 flexbox is used to make the elements behave predictably when they are used with different screen sizes and different display device
- It is mainly used to make CSS3 capable to change its item's width and height to best fit for all available spaces
- The CSS3 flexbox contains flex containers and flex items
  - **Flex container:** the flex container specifies the properties of the parent. It is declared by setting the display property of an element to either flex or inline-flex
  - **Flex items:** the flex items specify properties of the children. There may be one or more flex items inside a flex container
- Everything outside a flex container and inside a flex item is considered as usual



# CSS3 Flexbox Properties



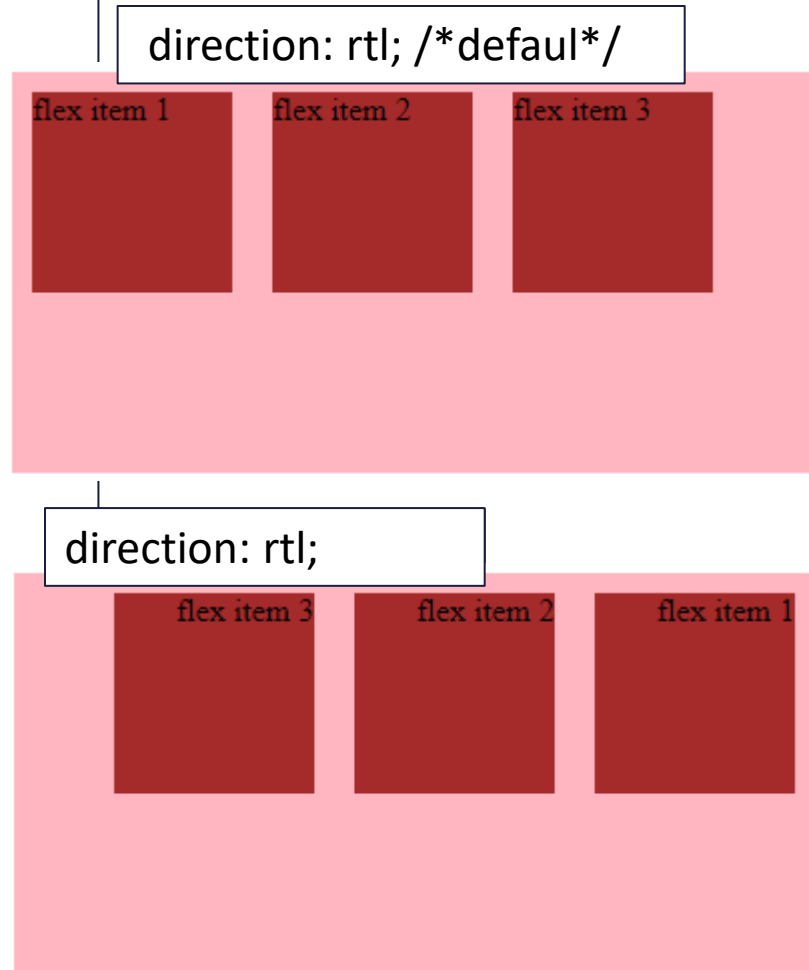
property	description
display	it is used to specify the type of box used for an html element.
flex-direction	it is used to specify the direction of the flexible items inside a flex container.
justify-content	it is used to align the flex items horizontally when the items do not use all available space on the main-axis.
align-items	it is used to align the flex items vertically when the items do not use all available space on the cross-axis.
flex-wrap	it specifies whether the flex items should wrap or not, if there is not enough room for them on one flex line.
align-content	it is used to modify the behavior of the flex-wrap property. it is similar to align-items, but instead of aligning flex items, it aligns flex lines.
flex-flow	it specifies a shorthand property for flex-direction and flex-wrap.
order	it specifies the order of a flexible item relative to the rest of the flex items inside the same container.
align-self	it is used on flex items. it overrides the container's align-items property.
flex	it specifies the length of a flex item, relative to the rest of the flex items inside the same container.

```

..
.flex-container {
 display: -webkit-flex;
 direction: rtl;
 display: flex;
 width: 400px;
 height: 200px;
 background-color: lightpink;
}
.flex-item {
 background-color: brown;
 width: 100px;
 height: 100px;
 margin: 10px;
}
....
<body>
<div class="flex-container">
 <div class="flex-item">flex item 1</div>
 <div class="flex-item">flex item 2</div>
 <div class="flex-item">flex item 3</div>
</div>...

```

# CSS3 Flexbox example



# CSS flex property



- The flex property in CSS is shorthand for flex-grow, flex-shrink, and flex-basis. It only works on the flex-items
- flex property is used to set how a flex-item will shrink or grow to fit in the space
- The flex property can be specified by one, two, or three values
  - **one-value:** can be specified by a number or keywords such as none, auto, or initial
  - **two-value:** the first value must be a number (used as flex-grow), the second value can either be a number (used for flex-shrink) or a valid width value (used as flex-basis)
  - **three-value:** then the values must follow the order, a number for the flex-grow, a number for the flex-shrink, and a valid width value for flex-basis
- Syntax
  - flex: flex-grow flex-shrink flex-basis | auto | none | initial | inherit;
- **flex-grow:** specifies how much the item will grow compared to the other flexible-items
- **flex-shrink:** specifies how much the item will shrink compared to the other flexible-items
- **flex-basis:** It is used to set the length of the flex-item
- **auto:** This value of the flex property is equivalent to 1 1 auto
- **none:** This value of the flex property is equivalent to 0 0 auto

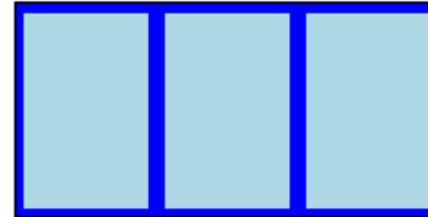
```
.container { width: 200px;
height: 100px;
border: 1px solid black;
display: flex;
margin: 15px;
background-color: blue; }
.flex-item{ flex: 1; // unitless number: flex-grow }
.flex-item1{ flex: 0 50px; // flex-grow, flex-basis }
.flex-item2{ flex: 2 2; // flex-grow, flex-shrink }
.flex-item, .flex-item1, .flex-item2 {
background-color: lightblue;
margin: 4px; }
```

```
.....
<h3> flex: 1; </h3>
<div class = "container">
 <div class = "flex-item"></div>
 <div class = "flex-item"></div>
 <div class = "flex-item"></div>
</div>
<h3> flex: 0 50px; </h3>
<div class = "container">
 <div class = "flex-item1"></div>
 <div class = "flex-item1"></div>
 <div class = "flex-item1"></div>
</div>
<h3> flex: 2 2; </h3>
<div class = "container">
 <div class = "flex-item2"></div>
 <div class = "flex-item2"></div>
 <div class = "flex-item2"></div>
</div>...
```

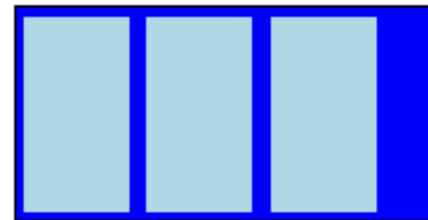
# CSS flex example



**flex: 1;**



**flex: 0 50px;**



**flex: 2 2;**



```
.container { width: 200px;
height: 100px;
border: 1px solid black;
display: flex;
margin: 15px;
background-color: blue; }
.flex-item{ flex: auto; }
.flex-item1{ flex: 0 1 30px;}
.flex-item, .flex-item1{
background-color: lightblue;
margin: 4px; }
```

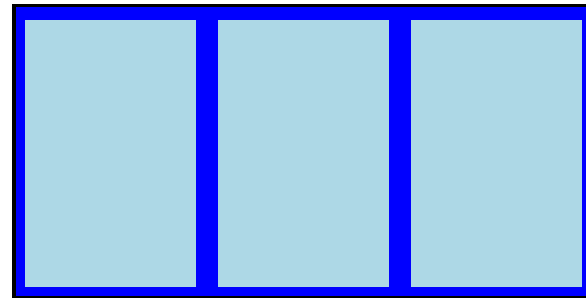
.....

```
<h3> flex: auto; </h3>
<div class = "container">
<div class = "flex-item"></div>
<div class = "flex-item"></div>
<div class = "flex-item"></div>
</div>
<h3> flex: 0 1 30px; </h3>
<div class = "container">
<div class = "flex-item1"></div>
<div class = "flex-item1"></div>
<div class = "flex-item1"></div>
</div>
```

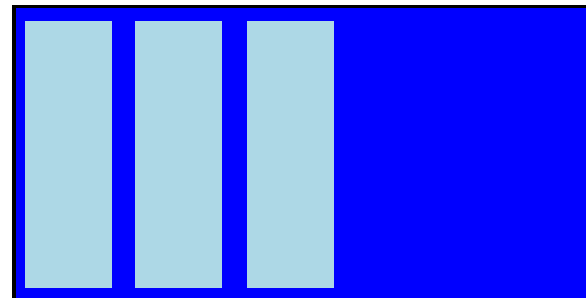
# CSS flex example



**flex: auto;**



**flex: 0 1 30px;**



# CSS flex-basis property



- Syntax
  - flex-basis: auto | width | initial | inherit;
- **auto**: It is the default value. This value sets the item's width equal to the value of its width property, if defined
- **width**: This value is defined using relative or absolute units

```
.container { display: flex; background-color: lightblue; }
.flex-item {
 background-color: white;
 text-align: center;
 line-height: 40px;
 font-size: 25px;
 margin: 5px; }
</style>
```

## Example of the flex-basis property

A horizontal bar with a light blue background is divided into five white rectangular segments. Each segment contains a text value: 'auto', 'initial', 'inherit', '150px', and 'auto'. The segments are of varying widths, with '150px' being the widest and 'auto' being the narrowest.

auto initial inherit 150px auto

```
.....
<div class = "container">
<div class = "flex-item" style = "flex-basis: auto;"> auto </div>
<div class = "flex-item" style = "flex-basis: initial;"> initial </div>
<div class = "flex-item" style = "flex-basis: inherit;"> inherit </div>
<div class = "flex-item" style = "flex-basis: 150px;"> 150px </div>
<div class = "flex-item" style = "flex-basis: auto"> auto </div>
</div>....
```

```
.container { height: 100px;
border: 2px solid red;
display: flex;
background-color: blue; }
```

```
.container div{ padding-top: 25px; flex-basis: 100px; }
.flex-item{ text-align: center; font-size: 25px; }
.container div:nth-of-type(1) { flex-basis: 50%; }
.container div:nth-of-type(3) {flex-basis: 200px; }
.container div:nth-of-type(5) { flex-basis: 7em; }
```

.....

<h1>The flex-basis Property</h1>

```
<div class = "container">
```

```
<div class = "flex-item" style= "background-color:
lightblue;"> 50%</div>
```

```
<div class = "flex-item" style= "background-color:
yellow;"> 100px</div>
```

```
<div class = "flex-item" style= "background-color:
pink;"> 200px </div>
```

```
<div class = "flex-item" style= "background-color:
orange;">100px </div>
```

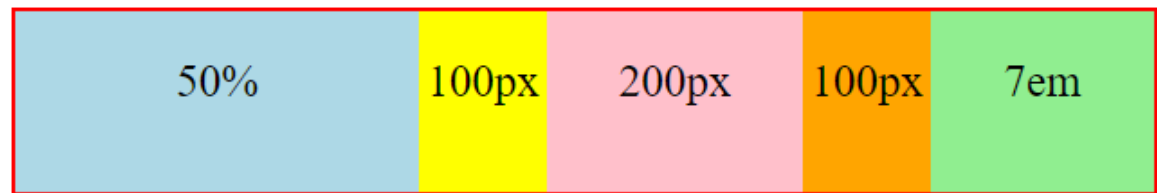
```
<div class = "flex-item" style= "background-color:
lightgreen;">7em</div>
```

```
</div>..
```

# CSS flex-basis example

AASTU

## The flex-basis Property



# CSS flex-grow property

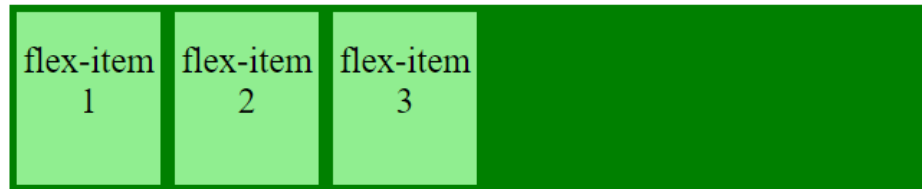
- The CSS flex-grow property specifies how much space a flex item should take if there is available space
  - Example
    - If we set flex-grow to 1 for all items, each item will set to an equal size in the container
    - If we give any of the items a value of 2, then the corresponding item will take up space twice than the others
- Syntax `flex-grow: number | initial | inherit;`



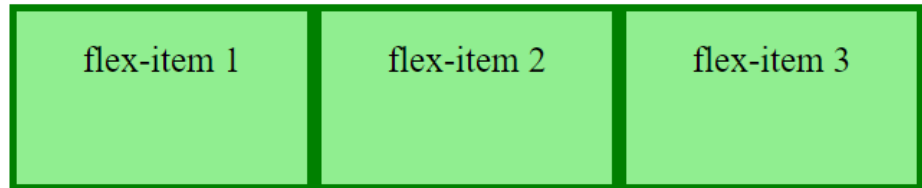
```
.container { display: -webkit-flex; display: flex;
background-color: green; }
.flex-item { background-color: lightgreen;
text-align: center; font-size: 25px;
width: 100px; height: 100px;
padding-top: 20px; margin: 5px; }
```

```
....
<h1> flex-grow: 0; </h1>
<div class="container">
<div class="flex-item" style = "flex-grow: 0;"> flex-item 1 </div>
<div class="flex-item" style = "flex-grow: 0;"> flex-item 2 </div>
<div class="flex-item" style = "flex-grow: 0;"> flex-item 3 </div>
</div>
<h1> flex-grow: 1; </h1>
<div class="container">
<div class="flex-item" style = "flex-grow: 1;"> flex-item 1 </div>
<div class="flex-item" style = "flex-grow: 1;"> flex-item 2 </div>
<div class="flex-item" style = "flex-grow: 1;"> flex-item 3 </div>
</div>
</div>...
```

## **flex-grow: 0;**



## **flex-grow: 1;**



```
.container { display: -webkit-flex; display: flex;
background-color: green; margin: 20px; }
```

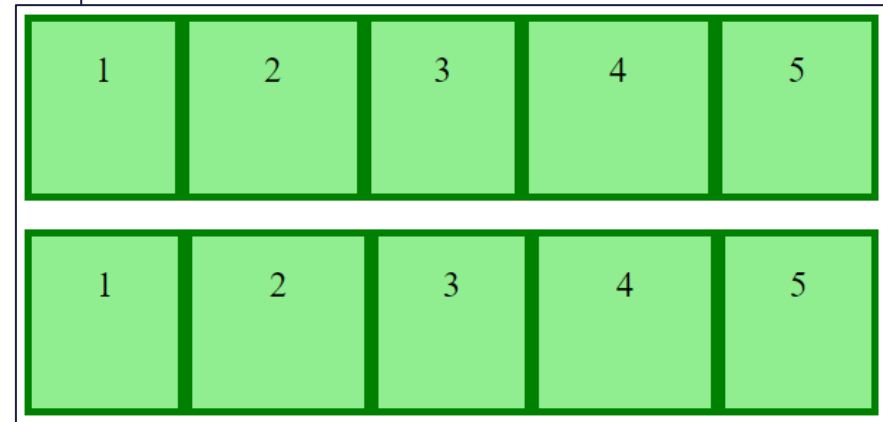
```
.flex-item {
background-color: lightgreen;
text-align: center;
font-size: 25px;
width: 100px; height: 100px;
padding-top: 20px; margin: 5px;
}
```

...

```
<div class="container">
<div class="flex-item" style = "flex-grow: 0;"> 1 </div>
<div class="flex-item" style = "flex-grow: 4;"> 2 </div>
<div class="flex-item" style = "flex-grow: 0;"> 3 </div>
<div class="flex-item" style = "flex-grow: 6;"> 4 </div>
<div class="flex-item" style = "flex-grow: 1;"> 5 </div>
</div>
```

```
<div class="container">
<div class="flex-item" style = "flex-grow: 1;"> 1 </div>
<div class="flex-item" style = "flex-grow: 9;"> 2 </div>
<div class="flex-item" style = "flex-grow: 1;"> 3 </div>
<div class="flex-item" style = "flex-grow: 9;"> 4 </div>
<div class="flex-item" style = "flex-grow: 1;"> 5 </div>
</div>...
```

# CSS flex-grow example



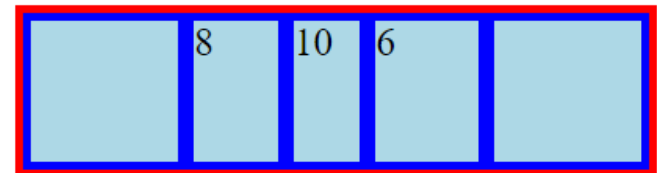
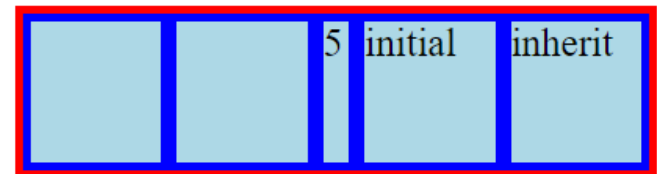
# CSS flex-shrink property

- The CSS flex-shrink property specifies how much an item will shrink than the other items of the container
- Example
  - The flex-item with the flex-shrink: 2; will shrink twice than the flex-shrink: 1;
  - The higher the flex-shrink value causes the item to shrink more than the others.
- **Syntax** - flex-shrink: number | initial | inherit;



```
.container { width: 400px; height: 100px;
border: 5px solid red; display: flex; margin: 30px;
background-color: blue; }
.flex-item { background-color: lightblue;
font-size: 25px; margin: 5px;
flex-grow: 1; flex-shrink: 1; flex-basis: 100px; } ...
<div class="container">
<div class = "flex-item"></div>
<div class = "flex-item"></div>
<div class = "flex-item" style = "flex-shrink: 5;"> 5 </div>
<div class = "flex-item" style = "flex-shrink: initial;"> initial </div>
<div class = "flex-item" style = "flex-shrink: inherit;"> inherit </div>
</div>
<div class="container">
<div class = "flex-item"></div>
<div class = "flex-item" style = "flex-shrink: 8;"> 8 </div>
<div class = "flex-item" style = "flex-shrink: 10;"> 10 </div>
<div class = "flex-item" style = "flex-shrink: 6;"> 6 </div>
<div class = "flex-item"></div>
</div>.....
```

## Example of the flex-shrink property

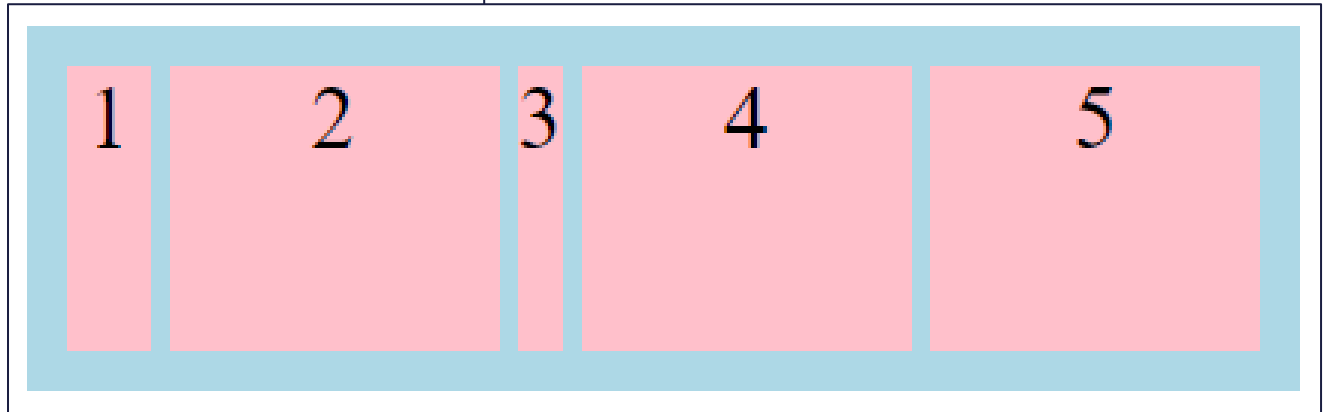


# CSS flex-shrink example

```
.container {
width: 400px; height: 100px;
padding: 10px; display: flex;
background-color: lightblue;
}
```

```
.flex-item {
background-color: pink;
margin: 3px;
text-align: center;
font-size: 30px;
flex-basis: 150px;
flex-grow: 0;
}
```

```
....
<div class="container">
<div class="flex-item" style = "flex-shrink: 3;">
1 </div>
<div class="flex-item" > 2 </div>
<div class="flex-item" style = "flex-shrink: 5;" >
3 </div>
<div class="flex-item" > 4 </div>
<div class="flex-item" > 5 </div>
</div>..
```



# CSS flex-flow property



- This CSS property is shorthand for **flex-direction** and **flex-wrap** properties
- Syntax
  - flex-flow: flex-direction flex-wrap | initial | inherit;

Values	Description
<b>flex-direction</b>	The flex-direction property is used to set the direction of the flexible items inside the flex container. Its default value is <b>row</b> (left-to-right, top-to-bottom). The possible values of this property are <b>row</b> , <b>row-reverse</b> , <b>column</b> , and <b>column-reverse</b> .
<b>flex-wrap</b>	The flex-wrap property specifies if the flex-items should wrap or not, in case of not enough space for them on one flex line. Its default value is <b>nowrap</b> . The possible values of this property are <b>nowrap</b> , <b>wrap</b> , and <b>wrap-reverse</b> .

```

.mainrow{ width: 400px; height: 50px; border: 5px solid red; }
.maincol{ width: 100px; height: 200px; border: 5px solid red; }
#row { flex-flow: row nowrap; }
#rowrev { flex-flow: row-reverse nowrap; }
#col { flex-flow: column nowrap; }
#colrev { flex-flow: column-reverse nowrap; }
div { width: 100px; height: 50px; display: flex; font-size: 20px; }
..... <h2> flex-flow: row nowrap;</h2>

```

```

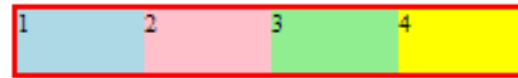
<div id= "row" class = "mainrow">
<div style="background-color: lightblue;"> 1 </div>
<div style="background-color: pink;"> 2 </div>
<div style="background-color: lightgreen"> 3 </div>
<div style="background-color: yellow;"> 4 </div>
</div> <h2> flex-flow: row-reverse nowrap;</h2>
<div id= "rowrev" class = "mainrow" >
<div style="background-color: lightblue;"> 1 </div>
<div style="background-color: pink;"> 2 </div>
<div style="background-color: lightgreen"> 3 </div>
<div style="background-color: yellow;"> 4 </div>
</div> <h2> flex-flow: column nowrap;</h2>
<div id="col" class = "maincol">
<div style="background-color: lightblue;"> 1 </div>
<div style="background-color: pink;"> 2 </div>
<div style="background-color: lightgreen"> 3 </div>
<div style="background-color: yellow;"> 4 </div>
</div> <h2> flex-flow: column-reverse nowrap;</h2>
<div id="colrev" class = "maincol">
<div style="background-color: lightblue;"> 1 </div>
<div style="background-color: pink;"> 2 </div>
<div style="background-color: lightgreen"> 3 </div>
<div style="background-color: yellow;"> 4 </div> </div>

```

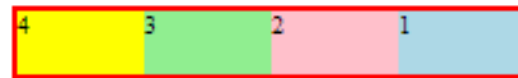
# CSS flex-flow example

MASTU

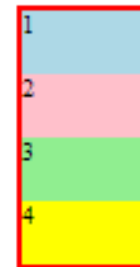
**flex-flow: row nowrap;**



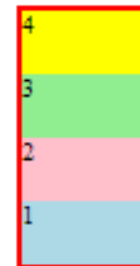
**flex-flow: row-reverse nowrap;**



**flex-flow: column nowrap;**



**flex-flow: column-reverse nowrap;**



```
.mainrow{ width: 200px; height: 100px; border: 5px solid red; }
.maincol{ width: 200px; height: 150px; text-align: left; border: 5px solid red; }
```

```
#row { flex-flow: row wrap; }
```

```
#rowrev { flex-flow: row-reverse wrap; }
```

```
#col { flex-flow: column wrap; }
```

```
#colrev { flex-flow: column-reverse wrap; }
```

```
div { width: 100px; height: 50px; display: flex;
font-size: 20px; font-weight: bold; }
```

```
... <h2> flex-flow: row wrap;</h2>
```

```
<div id= "row" class= "mainrow">
```

```
<div style="background-color: lightblue;"> 1 </div>
```

```
<div style="background-color: pink;"> 2 </div>
```

```
<div style="background-color: lightgreen"> 3 </div>
```

```
<div style="background-color: yellow;"> 4 </div>
```

```
</div> <h2> flex-flow: row-reverse wrap;</h2>
```

```
<div id= "rowrev" class= "mainrow" >
```

```
<div style="background-color: lightblue;"> 1 </div>
```

```
<div style="background-color: pink;"> 2 </div>
```

```
<div style="background-color: lightgreen"> 3 </div>
```

```
<div style="background-color: yellow;"> 4 </div>
```

```
</div> <h2> flex-flow: column wrap;</h2>
```

```
<div id="col" class= "maincol">
```

```
<div style="background-color: lightblue;"> 1 </div>
```

```
<div style="background-color: pink;"> 2 </div>
```

```
<div style="background-color: lightgreen"> 3 </div>
```

```
<div style="background-color: yellow;"> 4 </div>
```

```
</div> <h2> flex-flow: column-reverse wrap;</h2>
```

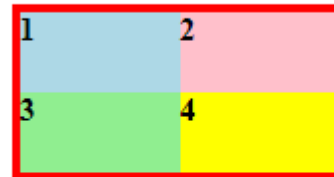
```
<div id="colrev" class= "maincol">
```

```
<div style="background-color: lightblue;"> 1 </div>....
```

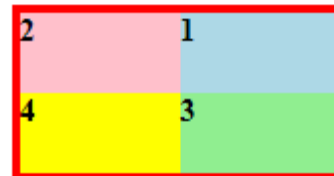
# CSS flex-flow example

using wrap

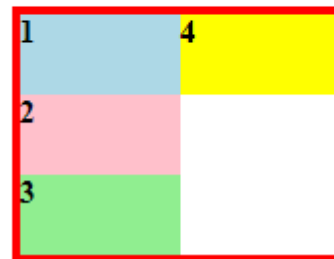
**flex-flow: row wrap;**



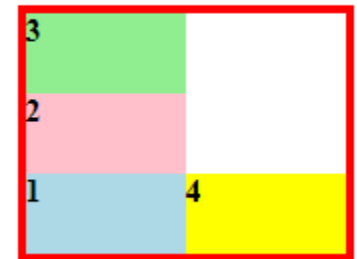
**flex-flow: row-reverse wrap;**



**flex-flow: column wrap;**



**flex-flow: column-reverse wrap;**



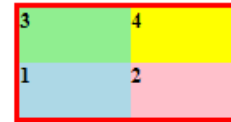
```
.mainrow{ width: 200px; height: 100px; border: 5px solid red;}
.maincol{ width: 200px; height: 150px; text-align: left; border: 5px solid red; }
#row { flex-flow: row wrap-reverse; }
#rowrev { flex-flow: row-reverse wrap-reverse; }
#col { flex-flow: column wrap-reverse; }
#colrev { flex-flow: column-reverse wrap-reverse; }
div { width: 100px; height: 50px; display: flex; font-size: 20px; font-weight:
```

```
bold; }
.....<h2> flex-flow: row wrap-reverse;</h2>
<div id= "row" class = "mainrow">
<div style="background-color: lightblue;"> 1 </div>
<div style="background-color: pink;"> 2 </div>
<div style="background-color: lightgreen"> 3 </div>
<div style="background-color: yellow;"> 4 </div>
</div> <h2> flex-flow: row-reverse wrap-reverse;</h2>
<div id= "rowrev" class = "mainrow" >
<div style="background-color: lightblue;"> 1 </div>
<div style="background-color: pink;"> 2 </div>
<div style="background-color: lightgreen"> 3 </div>
<div style="background-color: yellow;"> 4 </div>
</div> <h2> flex-flow: column wrap-reverse;</h2>
<div id="col" class = "maincol">
<div style="background-color: lightblue;"> 1 </div>
<div style="background-color: pink;"> 2 </div>
<div style="background-color: lightgreen"> 3 </div>
<div style="background-color: yellow;"> 4 </div>
</div> <h2> flex-flow: column-reverse wrap-reverse;</h2>
<div id="colrev" class = "maincol">
<div style="background-color: lightblue;"> 1 </div>
<div style="background-color: pink;"> 2 </div>
<div style="background-color: lightgreen"> 3 </div>..
```

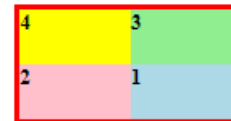
# CSS flex-flow example

Using **wrap-reverse** value of the **flex-wrap** & all the possible values of the **flex-direction** property

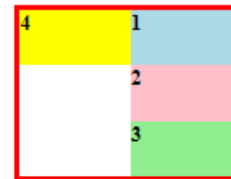
**flex-flow: row wrap-reverse;**



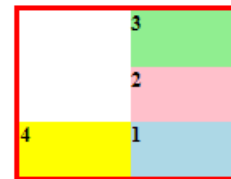
**flex-flow: row-reverse wrap-reverse;**



**flex-flow: column wrap-reverse;**



**flex-flow: column-reverse wrap-reverse;**



# Reading Assignment



- Animation
- Translate
- Transition
- Pseudo-classes
- Pseudo-elements
- Gradients
- Masking
- @keyframes
- Units
- Combinators
- Minify
- 2D Transforms
- 3D Transforms

# CSS media queries



- Media queries allow you to apply CSS styles depending on a **device's general type** (such as print vs. screen) or other characteristics such as **screen resolution** or **browser viewport width**
- Media queries are used for the following:
  - To conditionally apply styles with the CSS @media and @import at-rules.
  - To target specific media for the <style>, <link>, <source>, and other HTML elements with the media= attribute.
  - To test and monitor media states using the Window.matchMedia() and MediaQueryList.addListener() JavaScript methods
- A media query is composed of an optional **media type** and **any number of media feature expressions**
- **Media types** define the broad category of device for which the media query applies: all, print, screen. The type is optional (assumed to be all) except when using the not or only logical operators.
- **Media features** describe a specific characteristic of the user agent, output device, or environment
  - E.g. any-hover, any-pointer, aspect-ratio, color, display-mode, height, width...
- For example, the hover feature allows a query to test against whether the device supports hovering over elements
- **Logical operators** can be used to compose a complex media query: **not**, **and**, **or** and **only**

# CSS Media Query syntax/example



- **Targeting media types:**
  - `@media print { /* ... */ }`
  - You can also target multiple devices
    - `@media screen, print { /* ... */ }`
- **Targeting media features:**
  - `@media (hover: hover) { /* ... */ }`
- Many media features are range features, which means they can be prefixed with "min-" or "max-" to express "minimum condition" or "maximum condition" constraints
  - `@media (max-width: 1250px) { /* ... */ }`
- **Combining multiple types or features:**
  - `@media (min-width: 30em) and (orientation: landscape) { /* ... */ }`
  - To limit the styles to devices with a screen, you can chain the media features to the screen media type:
  - `@media screen and (min-width: 30em) and (orientation: landscape) { /* ... */ }`
- **Testing for multiple queries:** you can use a comma-separated list to apply styles when the user's device matches any one of various media types, features, or states
  - `@media (min-height: 680px), screen and (orientation: portrait) { /* ... */ }`

# CSS Media Query syntax/example



- **Inverting a query's meaning**

- `@media not all and (monochrome) { /* ... */ }`
- This means that the above query is evaluated like this:
  - `@media not (all and (monochrome)) { /* ... */ }`
- It wouldn't be evaluated like this:
  - `@media (not all) and (monochrome) { /* ... */ }`
- As another example, the following media query:
  - `@media not screen and (color), print and (color) { /* ... */ }`
- This means that the above query is evaluated like this:
  - `@media (not (screen and (color))), print and (color) { /* ... */ }`

- The **only** keyword prevents older browsers that do not support media queries with media features from applying the given styles

- It has no effect on modern browsers
- `@media only screen and (color) { /* ... */ }`

# CSS Media Query Syntax in Level 4



- `@media (max-width: 30em) { /* ... */ }` in level 4  
**`@media (width <= 30em) { /* ... */ }`**
- `@media (min-width: 30em) and (max-width: 50em) { /* ... */ }` in level 4 **`@media (30em <= width <= 50em) { /* ... */ }`**
- **Negating a feature with not:** Using `not()` around a media feature negates that feature in the query
  - `@media (not(hover)) { /* not(hover) would match if the device had no hover capability */ }`
- **Testing for multiple features with or**
  - `@media (not (color)) or (hover) { /* ... */ }`



- The diagram illustrates a three-column page layout. At the top is a blue header bar with the text "JavaTpoint" in white. Below the header is the main content area, which is divided into three columns. The leftmost column is labeled "Left Page" and is light blue. The middle column is labeled "Body Page" and is light orange; it contains the text "Page Content goes here" repeated ten times. The rightmost column is labeled "Right Page" and is light blue. At the bottom is a blue footer bar with the text "Footer" in white.

